Basi di Dati

Progetto A.A. 2024/2025

SISTEMA DI GESTIONE DI TRASPORTO FERROVIARIO

0306680

CHIARA CARUMANI

Indice

1. Descrizione del Minimondo **2**

2. Analisi dei Requisiti **3**

3. Progettazione concettuale **8**

4. Progettazione logica **21**

5. Progettazione fisica **37**

1. Descrizione del Minimondo

Si vuole realizzare un sistema informativo per la gestione dell'operatività di un'azienda di trasporto pubblico ferroviario ad alta velocità per passeggeri. I treni gestiti dal servizio sono caratterizzati da una matricola (codice univoco numerico di quattro cifre). Ogni veicolo è anche associato ad una data di acquisto e ad uno storico di manutenzione. È inoltre di interesse mantenere informazioni legate alla marca e modello delle locomotrici e vagoni. Di ciascun treno è di interesse conoscere anche da quanti vagoni questo è composto. Sono memorizzati il numero di carrozze di prima e seconda classe, il numero massimo di passeggeri che possono viaggiare in ciascun vagone (con riferimento ai singoli posti).

Ciascuna tratta ha un insieme di fermate identificate dal nome della stazione, dalla città e dalla provincia in cui si trova. Inoltre, per ciascuna tratta, vengono mantenuti i capilinea di partenza ed arrivo. Ciascuna fermata è associata all'orario di arrivo e partenza previsti, eccezion fatta per i capilinea in cui uno solo dei due orari è disponibile. Ogni tratta viene viene coperta da un numero predefinito di treni, la cui associazione viene gestita dai gestori del servizio. I gestori possono, su base periodica, modificare il numero di treni operanti su ciascuna tratta.

I gestori del servizio associano anche un macchinista e un capotreno a ciascun treno. La gestione dei turni avviene da parte dei gestori del servizio su base mensile. Ogni lavoratore ha la possibilità di generare un report sui propri turni di lavoro, su base settimanale, riportante l'indicazione degli orari e dei treni in cui esso è coinvolto.

Si vuole anche realizzare una funzionalità di prenotazione di biglietto. All'atto di acquisto di un biglietto (identificato per tratta e cui viene associato un posto disponibile sul treno con l'aggiunta di un codice di prenotazione univoco) l'acquirente deve indicare il proprio nome, cognome data di nascita, codice fiscale e numero di carta di credito.

2. Analisi dei Requisiti

Identificazione dei termini ambigui e correzioni possibili

Linea	Termine	Nuovo termine	Motivo correzione
4,11	Associato	Si vuole memorizzare	Il termine “associato” potrebbe sembrare un’associazione
7	Carrozza	Vagone	Termine più specifico
14	Gestori	Gestori del servizio	Termine più specifico
2	Servizio	Sistema	Termine più specifico
6,7	Sono memorizzati	Per ogni treno sono memorizzati	Termine più specifico
3	Veicolo	Treno	Termine più specifico

Specifica disambiguata

Sistema di gestione di trasporto ferroviario

Si ha l'obiettivo di realizzare un sistema informativo per supportare la gestione operativa di un'azienda di trasporto pubblico ferroviario ad alta velocità per passeggeri. I treni gestiti all'interno del sistema sono identificati da una matricola, ovvero un codice numerico univoco di quattro cifre. Per ciascun treno si vuole memorizzare la data di acquisto e uno storico relativo alla manutenzione. È inoltre rilevante conservare le informazioni riguardanti la marca e il modello delle locomotrici e dei vagoni. Per ogni treno è importante conoscere anche il numero complessivo di vagoni di cui è composto. Vengono memorizzati il numero di vagoni di prima e seconda classe, nonché il numero massimo di passeggeri trasportabili in ciascun vagone, con riferimento ai singoli posti.

Ogni tratta è composta da un insieme di fermate, identificate tramite il nome della stazione, la città e la provincia di ubicazione. Per ciascuna tratta vengono inoltre memorizzati i capolinea di partenza e arrivo. Di ogni fermata si vuole memorizzare l'orario previsto di arrivo e partenza, ad eccezione dei capolinea per cui è disponibile solo uno dei due orari. Ogni tratta è servita da un numero definito di treni, la cui assegnazione è gestita dai responsabili del servizio. Questi ultimi possono modificare periodicamente il numero di treni attivi su ciascuna tratta.

I gestori del servizio assegnano anche un macchinista e un capotreno a ogni treno. La pianificazione dei turni viene effettuata su base mensile. Ogni lavoratore ha la possibilità di generare un report settimanale sui propri turni di lavoro, contenente le informazioni sugli orari e sui treni a cui è stato assegnato.

Si intende inoltre realizzare una funzionalità per la prenotazione dei biglietti. Al momento dell'acquisto di un biglietto (identificato per tratta e a cui viene associato un posto disponibile sul treno, identificato da un codice di prenotazione univoco), l'acquirente è tenuto a fornire il proprio nome, cognome, data di nascita, codice fiscale e numero di carta di credito.

Glossario dei Termini

Termine	Descrizione	Sinonimi	Collegamenti
Acquirente	Persona che acquista un biglietto	Utente	Prenotazione
Fermata	Punto intermedio o capolinea lungo una tratta dove il treno effettua una sosta programmata.	Stazione	Tratta
Lavoratore	Persona impiegata nel servizio ferroviario	Personale	Treno
Locomotrici	Unità motrice del treno, responsabile della sua propulsione.		Treno
Prenotazione	Procedura con cui un acquirente riserva un posto su un treno per una determinata tratta.	Biglietto	Tratta
Tratta	Percorso ferroviario definito tra una stazione di partenza e una di arrivo, con eventuali fermate intermedie.	Viaggio	Prenotazione, Fermate
Treno	Mezzo ferroviario utilizzato per il trasporto dei passeggeri	Veicolo	Vagone, Locomotrici, Turni, Tratta
Vagone	Carrozza collegata alla locomotrice, destinata al trasporto dei passeggeri, divise in prima e seconda classe	Carrozza	Treno

Raggruppamento dei requisiti in insiemi omogenei

Frasi relative a Acquirente
“Al momento dell’acquisto di un biglietto (identificato per tratta e a cui viene associato un posto disponibile sul treno, identificato da un codice di prenotazione univoco), l’acquirente è tenuto a fornire il proprio nome, cognome, data di nascita, codice fiscale e numero di carta di credito.”

Frasi relative a Fermata
"Ogni tratta è composta da un insieme di fermate, identificate tramite il nome della stazione, la città e la provincia di ubicazione." "Di ogni fermata si vuole memorizzare l’orario previsto di arrivo e partenza, ad eccezione dei capolinea per cui è disponibile solo uno dei due orari."

Frasi relative a Lavoratore
"Ogni lavoratore ha la possibilità di generare un report settimanale sui propri turni di lavoro, contenente le informazioni sugli orari e sui treni a cui è stato assegnato."

Frasi relative a Locomotrice
"Ogni lavoratore ha la possibilità di generare un report settimanale sui propri turni di lavoro, contenente le informazioni sugli orari e sui treni a cui è stato assegnato."

Frasi relative a Prenotazione
"Si intende inoltre realizzare una funzionalità per la prenotazione dei biglietti." "Al momento dell’acquisto di un biglietto (identificato per tratta e a cui viene associato un posto disponibile sul treno, identificato da un codice di prenotazione univoco), l’acquirente è tenuto a fornire il proprio nome, cognome, data di nascita, codice fiscale e numero di carta di credito."

Frasi relative a Tratta
“Ogni tratta è composta da un insieme di fermate, identificate tramite il nome della stazione, la città e la provincia di ubicazione.” “Per ciascuna tratta vengono inoltre memorizzati i capolinea di partenza e arrivo.” “Ogni tratta è servita da un numero definito di treni, la cui assegnazione è gestita dai responsabili del servizio.” “Questi ultimi possono modificare periodicamente il numero di treni attivi su ciascuna tratta.” “Al momento dell’acquisto di un biglietto (identificato per tratta e a cui viene associato un posto disponibile sul treno...)”

Frasi relative a Treno
“I treni gestiti all’interno del sistema sono identificati da una matricola, ovvero un codice numerico univoco di quattro cifre.” “Per ciascun treno si vuole memorizzare la data di acquisto e uno storico relativo alla manutenzione.” “Per ogni treno è importante conoscere anche il numero complessivo di vagoni di cui è composto.” “Ogni tratta è servita da un numero definito di treni, la cui assegnazione è gestita dai responsabili del servizio.” “Questi ultimi possono modificare periodicamente il numero di treni attivi su ciascuna tratta.” “I gestori del servizio assegnano anche un macchinista e un capotreno a ogni treno.” “Ogni lavoratore ha la possibilità di generare un report settimanale sui propri turni di lavoro, contenente le informazioni sugli orari e sui treni a cui è stato assegnato.” “Al momento dell’acquisto di un biglietto (identificato per tratta e a cui viene associato un posto disponibile sul treno, identificato da un codice di prenotazione univoco)...”

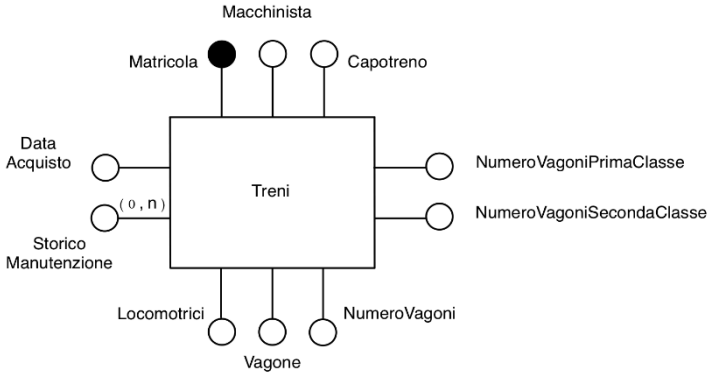
Frasi relative a Vagone
"Al momento dell’acquisto di un biglietto (identificato per tratta e a cui viene associato un posto disponibile sul treno, identificato da un codice di prenotazione univoco), l’acquirente è tenuto a fornire il proprio nome, cognome, data di nascita, codice fiscale e numero di carta di credito."

3. Progettazione concettuale

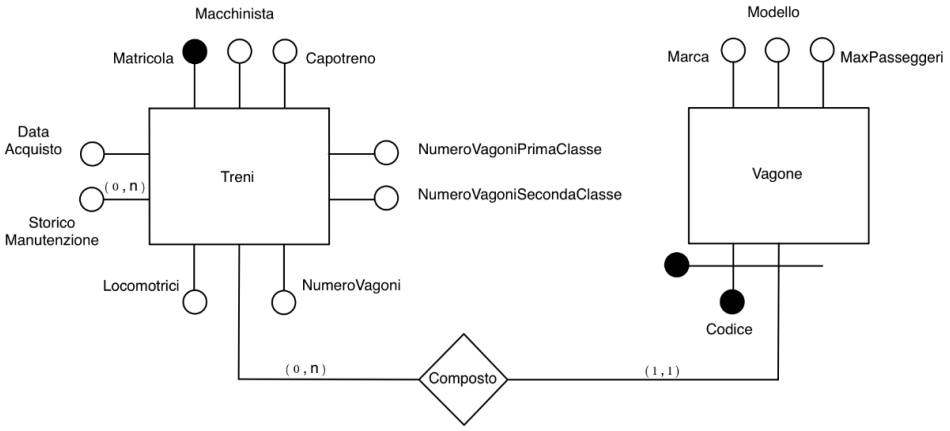
Costruzione dello schema E-R

Strutturiamo la progettazione concettuale della nostra base dati seguendo l'approccio inside-out, che prevede un sviluppo bottom-up dei concetti principali. I concetti secondari verranno poi definiti in relazione a quelli primari.

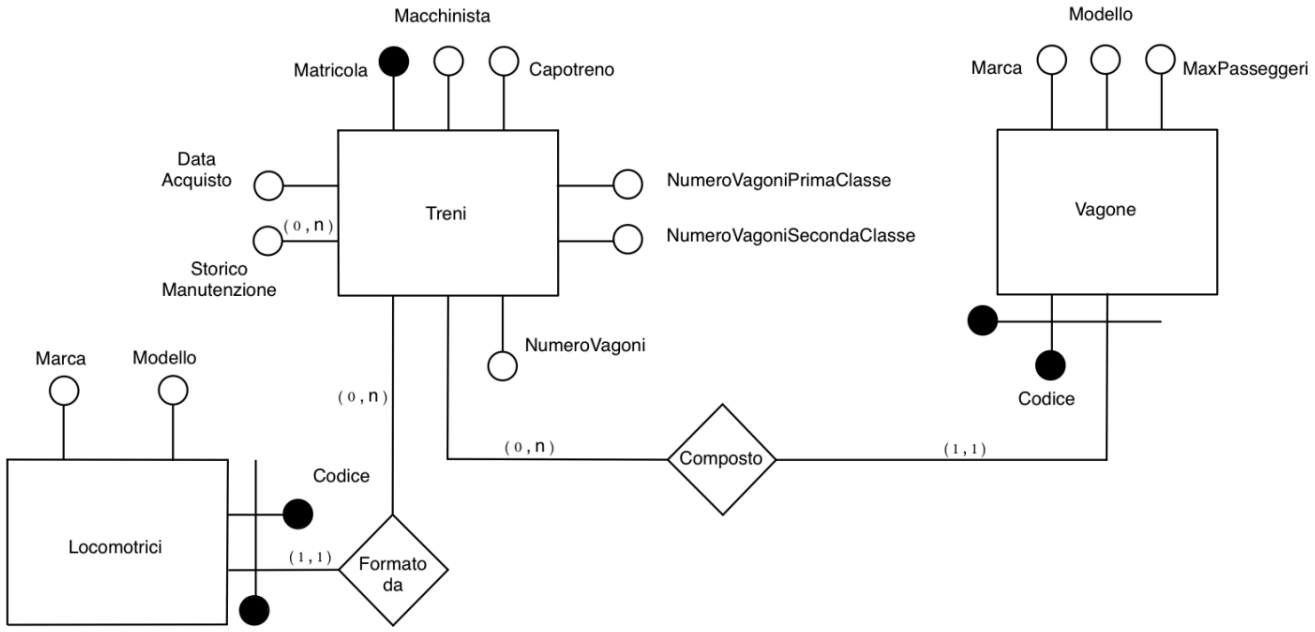
Il mini-mondo di riferimento introduce anzitutto l'entità *Treni*, identificata dall'attributo *matricola*. Per ciascun treno vengono registrati la *data di acquisto*, il *numero totale di vagoni*, il *numero di vagoni di prima classe* e *di seconda classe*, oltre a un attributo multivalore relativo allo *storico manutenzione*. Raggruppando queste caratteristiche, l'entità *Treni* è descritta dal seguente schema:



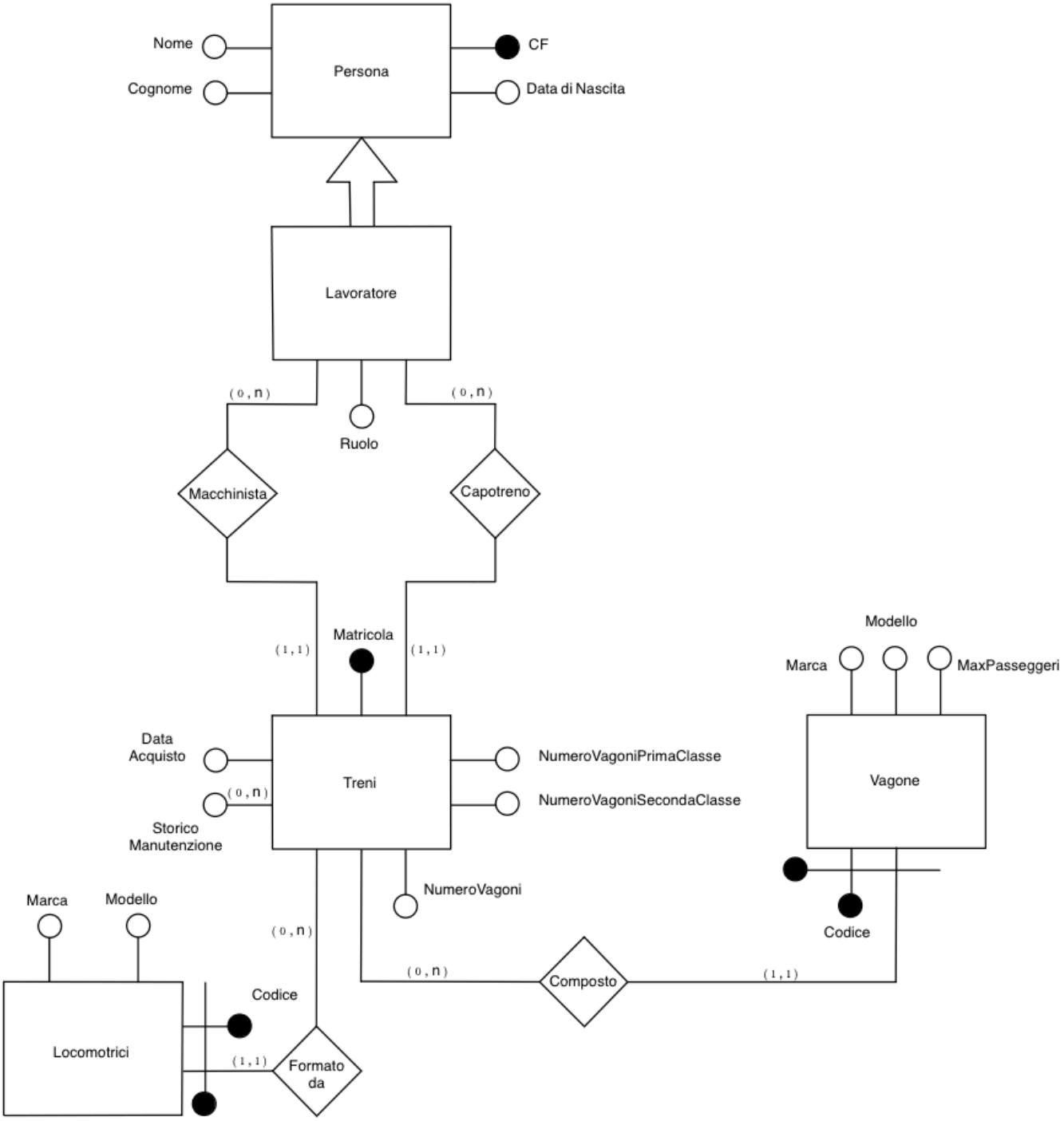
Dobbiamo mantenere diverse specifiche per i vagoni, come modello, marca e capacità massima di passeggeri, quindi adottiamo il pattern "Part of". Un treno può includere da 0 a n vagoni, ciascuno identificato da un codice univoco, poiché marca, modello e numero massimo di passeggeri non sono sufficienti per garantirne l'unicità. La relazione "composto" è di tipo uno a molti: un treno può avere più vagoni, mentre ogni vagone è associato a un solo treno. Raggruppando le specifiche che caratterizzano le entità Vagoni e Treni, otteniamo il seguente schema:



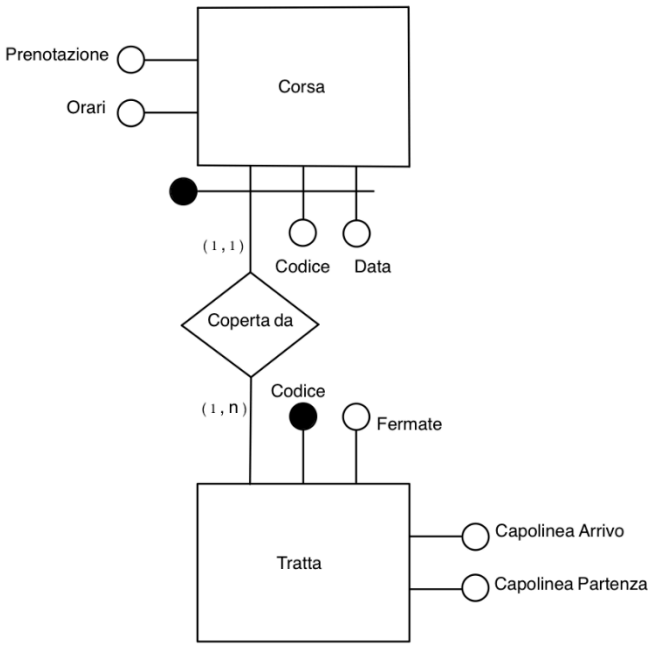
Successivamente, applichiamo il pattern "Part of" anche all'entità Locomotrici, poiché si desidera tenere traccia di specifiche come marca e modello. L'identificazione avviene tramite un codice, analogamente a quanto fatto per l'entità Vagone. Inoltre, l'unità Locomotrice è collegata all'entità Treno mediante una relazione uno a molti: un treno può includere da 0 a n locomotive, mentre ogni locomotiva deve essere associata a un treno. Raggruppando le specifiche che caratterizzano le entità Locomotrici e Treni, si ottiene il seguente schema:



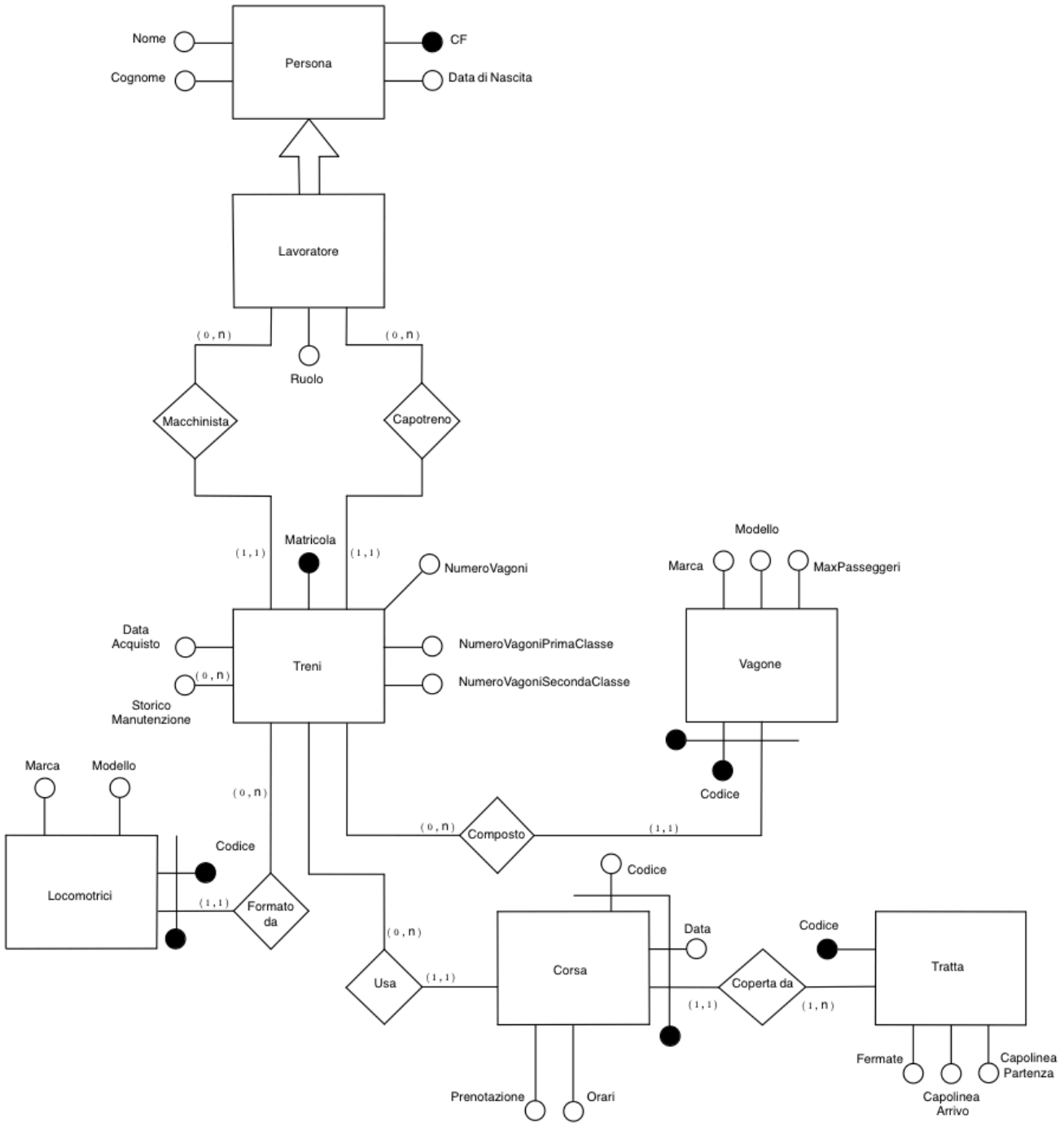
Ogni treno deve essere dotato di un macchinista e un capotreno, ciascuno con i propri attributi. Per questo motivo, l'attributo relativo a queste figure viene reificato nell'entità *Lavoratore*, poiché rappresenta un concetto chiave nel nostro Mini-mondo, e collegato a *Treno* tramite le relazioni "macchinista" e "capotreno". Si tratta di una generalizzazione parziale, in cui l'entità *Persone* funge da entità padre e *Lavoratore* da entità figlia, attraverso una relazione *IS-A*. L'entità *Persone* include gli attributi *data di nascita, cognome, nome* e viene identificata tramite *codice fiscale*.



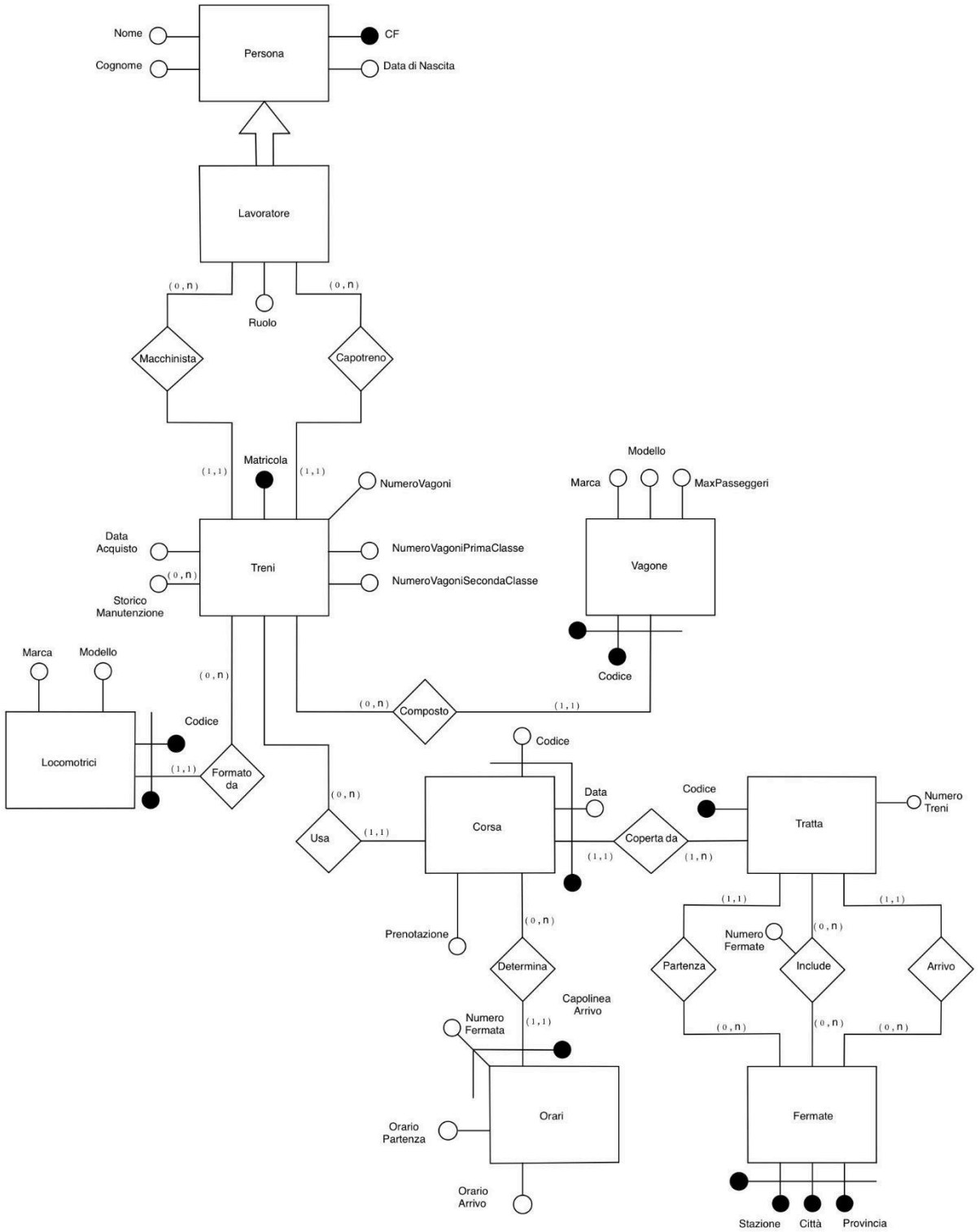
Ora introduciamo l'entità *tratta*, che include i campi capolinea di partenza e di arrivo. Tuttavia, queste informazioni non sono sufficienti, poiché possono esistere più tratte con gli stessi capolinea ma fermate diverse; per questo motivo, è necessario un codice identificativo. L'entità *tratta* segue il pattern "instance of" rispetto all'entità *corsa*, che rappresenta il viaggio effettivo del treno. La *corsa* ha come attributi la data, l'orario e la prenotazione ed è identificata univocamente dal codice, dalla data e dalla tratta percorsa.



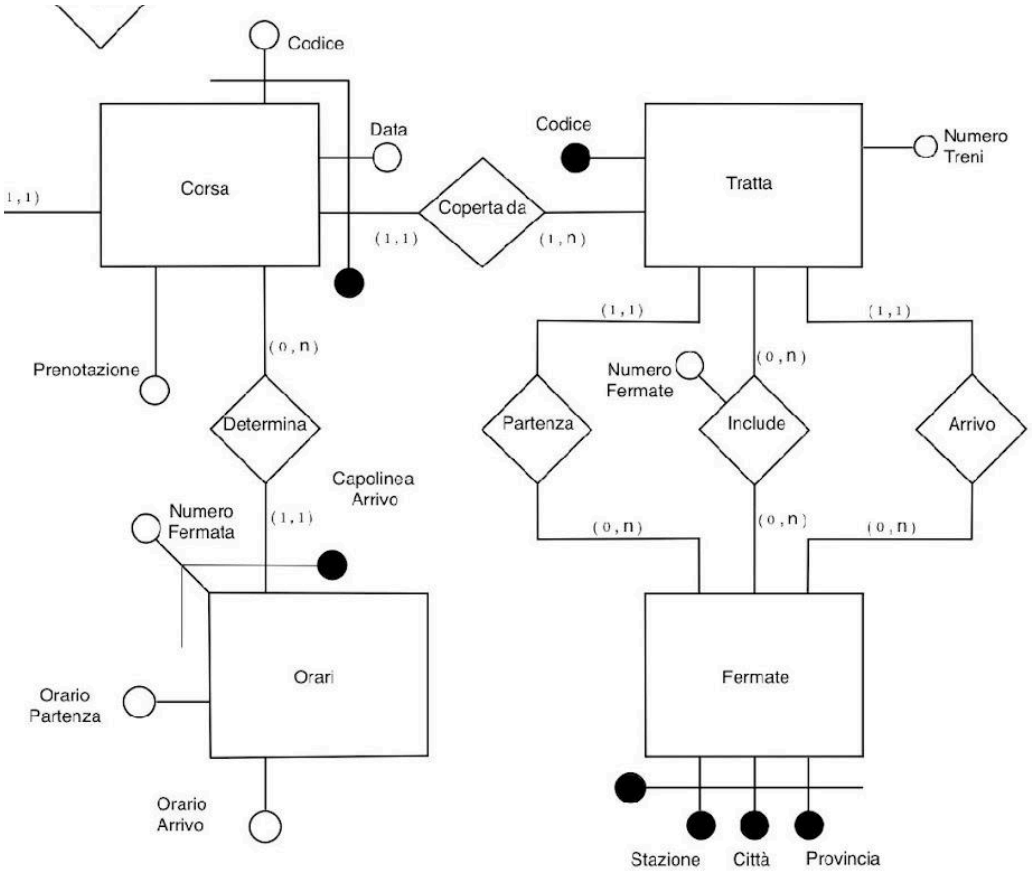
La corsa è collegata all'entità Treni attraverso un'associazione uno a molti, poiché un treno può coprire zero o più corse, mentre ogni tratta deve essere servita da un unico treno. Raggruppando le specifiche relative alle entità Tratta e Treni, si ottiene il seguente schema:



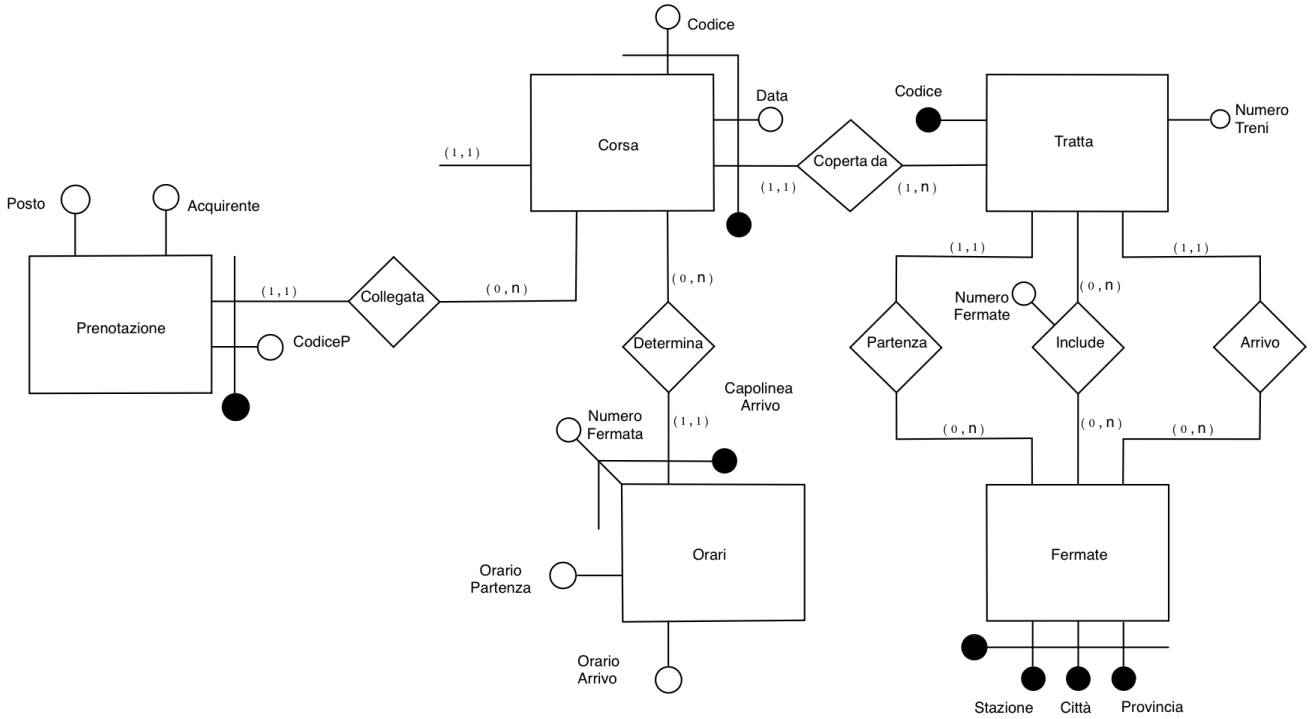
Successivamente, si reifica l'attributo *fermata* presente in *tratta*, creando l'entità *fermata*, identificata dalla tripletta di attributi *fermata, città* e *provincia*, in conformità al Minimondo di riferimento. Tra *tratte* e *fermate* esiste un'associazione molti a molti, poiché una *tratta* può includere 0, n *fermate* e una *fermata* può appartenere a 0, n *tratte*. Inoltre, aggiungo le associazioni *partenza* e *arrivo* per registrare i capolinea di ogni *tratta*. Nella relazione *include*, inserisco l'attributo *numero della fermata* per mantenere l'ordinamento numerico delle fermate lungo ogni tratta.



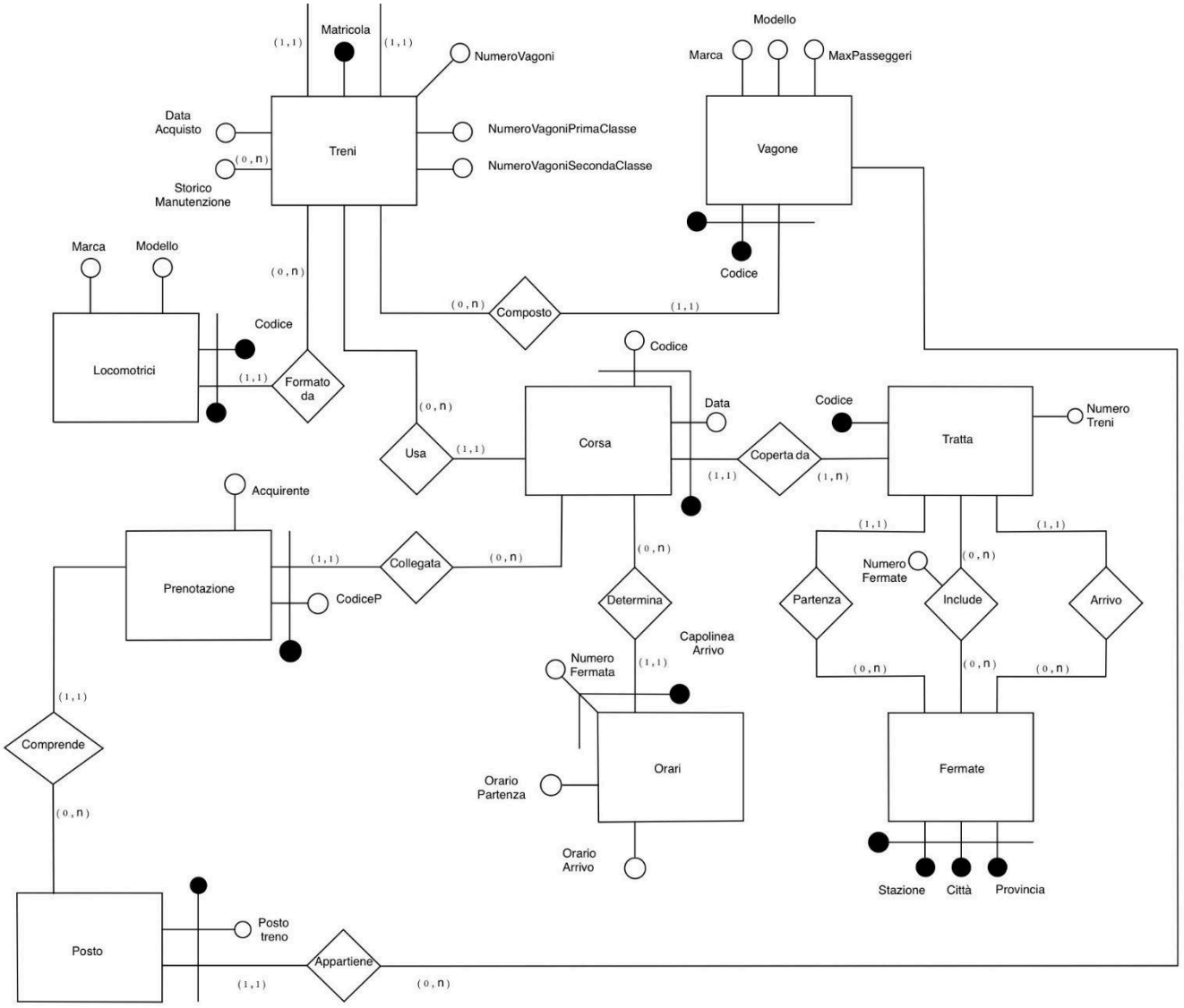
Reifico l'attributo *orari* presente in *corsa*, creando un'entità *orari* che registra l'orario di arrivo, di partenza e il numero della fermata effettuata dalla corsa in base alla tratta. L'entità è identificata tramite il numero della fermata e la relazione con *corsa*. Questa scelta progettuale nasce dal fatto che, in un giorno, possono esistere più corse sulla stessa tratta, con orari variabili per ciascuna. Inoltre, il numero della fermata nella tabella *orari* deve coincidere con quello presente nella relazione *include* tra *Tratta* e *Fermate*.



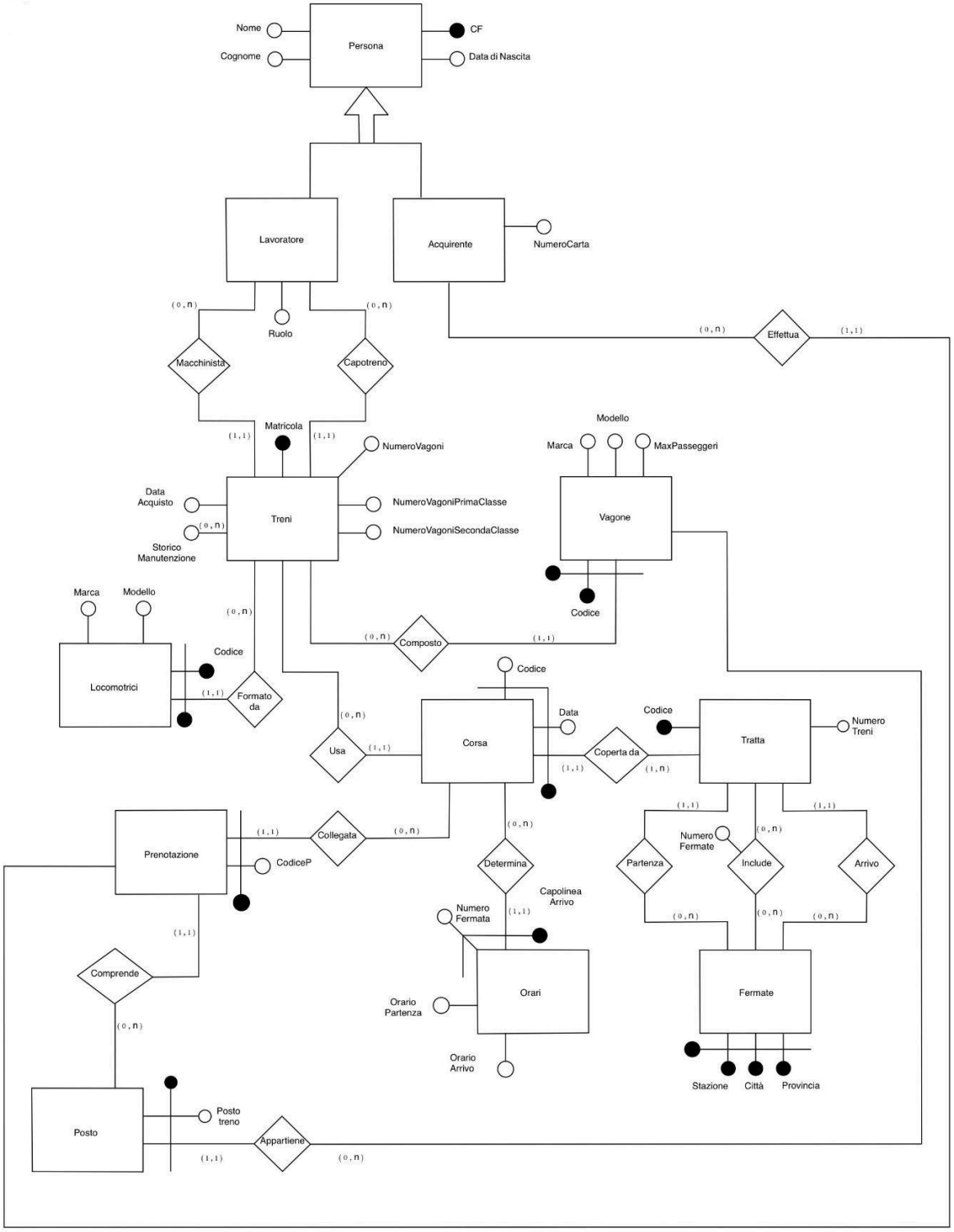
Della corsa reifico anche l'attributo prenotazione, creando l'entità omonima, identificata dal codice di prenotazione e dalla relazione con la corsa. Vengono memorizzati l'utente che effettua la prenotazione e il posto assegnato. La relazione con la corsa è di tipo "uno a n", poiché ogni prenotazione è associata a una sola corsa, mentre una corsa può avere da zero a più prenotazioni.



Successivamente, trasformo l'attributo "posto" di Prenotazione in un'entità chiamata Posto, la quale tiene traccia del posto a sedere. Tra le due entità si stabilisce una relazione "uno a molti", in quanto una prenotazione è associata a un solo posto, mentre un posto può essere legato a 0 o più prenotazioni. L'entità Posto è identificata dal posto stesso e dalla relazione con l'entità Vagoni, poiché ogni posto è associato a un unico vagone, mentre un vagone può avere 0 o più posti. Si ottiene così il seguente schema:

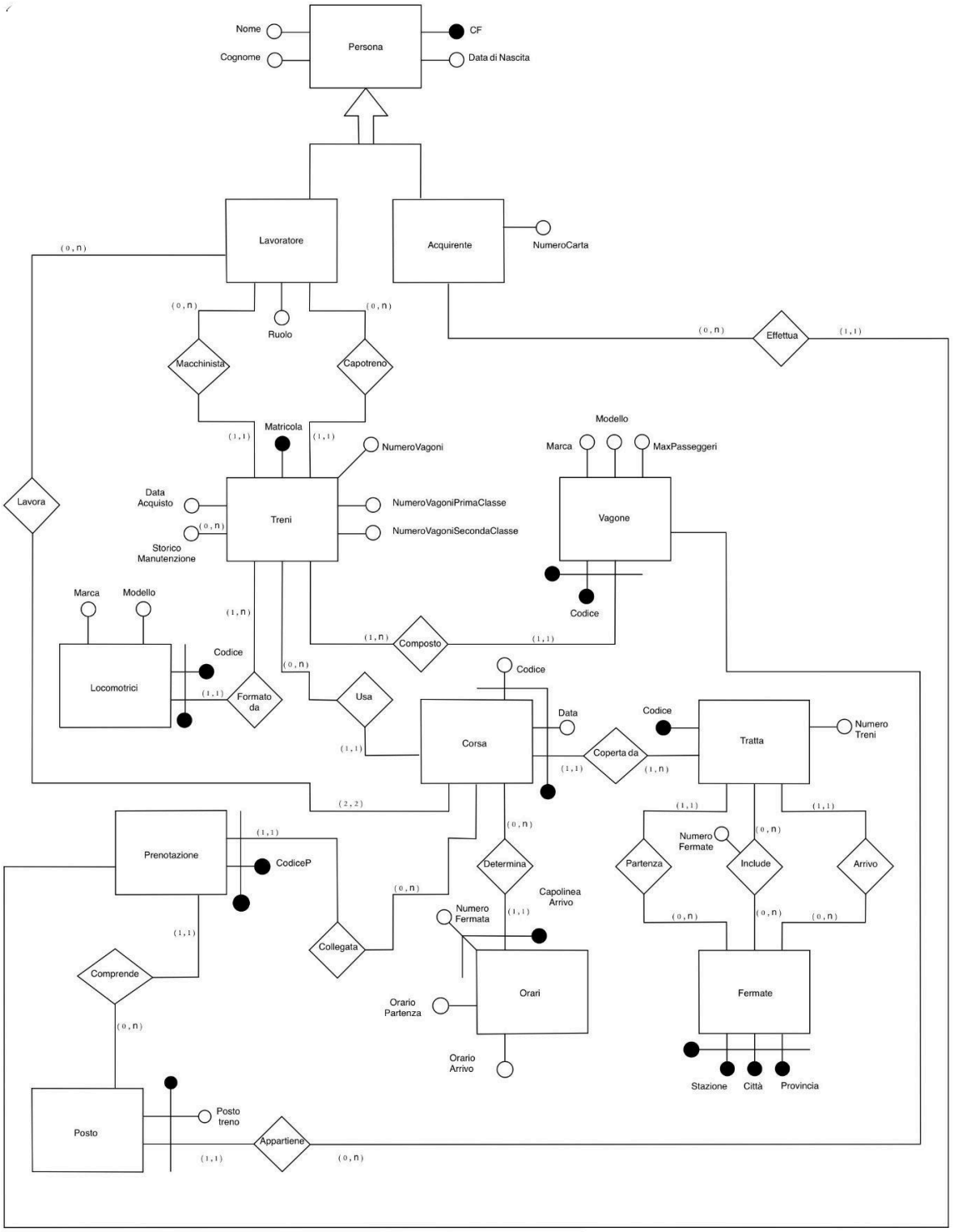


Come ultima modifica, trasformo l'attributo acquirente in una nuova entità, poiché è necessario registrare il nome, cognome, data di nascita, codice fiscale e numero di carta di credito. L'entità acquirente viene inserita come entità derivata dalla classe madre Persone, poiché condivide gli stessi attributi, con l'unica differenza dell'aggiunta della carta di credito. La relazione tra prenotazione e acquirente è di tipo "uno a molti", poiché ogni prenotazione deve essere legata a un acquirente, mentre un acquirente può effettuare 0 o più prenotazioni.



Infine, definisco l'ultima relazione tra lavoratore e corse, in quanto, come indicato dalle specifiche, l'orario lavorativo è strettamente connesso alle corse a cui partecipa il lavoratore. La relazione tra Lavoratore e Corse è di tipo "due a n", poiché un lavoratore può essere coinvolto in 0 o più corse, mentre ogni corsa deve prevedere la partecipazione di due lavoratori: un macchinista e un capotreno.

Integrazione finale



Regole aziendali

- L'orario di partenza da una fermata deve essere successivo all'orario di arrivo della stessa fermata.
- Il numero della carta di credito deve essere composto da sedici caratteri.
- Un macchinista o un capotreno non devono essere assegnati a più corse nello stesso orario di lavoro.
- Un treno non deve effettuare corse diverse negli stessi orari.
- La matricola di un treno deve essere un numero di quattro cifre.
- Per le fermate capolinea deve essere disponibile solo l'orario di arrivo o di partenza.
- Il numero di prenotazioni per una corsa non deve superare la capienza massima del treno.
- Ogni lavoratore deve essere associato esclusivamente al treno su cui presta servizio.
- Il numero della fermata registrato nell'entità "Orari" deve coincidere con il numero della fermata nella relazione "Include".
- Un treno non deve essere sottoposto a più interventi di manutenzione nello stesso giorno.
- I vagoni devono appartenere solo alla prima o alla seconda classe.

Dizionario dei dati

Entità	Descrizione	Attributi	Identificatori
Acquirenti	Racchiude le informazioni sul cliente che effettua la prenotazione.	NumeroCarta	CF
Corsa	La corsa reale fatta dal tremo.	CodiceCorsa, Data	CodiceCorsa, Data, CodiceTratta
Fermate	Dove si ferma il treno.	Stazione, Città, Provincia	Stazione, Città, Provincia
Lavoratori	L'insieme di tutte le persone che operano all'interno di un'organizzazione o che sono impegnate in un contesto lavorativo specifico.	Ruolo	CF
Locomotrici	Utilizzata per trainare treni composti da vagoni o carri ferroviari.	Marca, Modello	CodiceLoc, MatricolaTreno
Orari	Orari in cui vengono fatte le fermate.	NumFermata, OrarioPartenza, OrariArrivo	NumFermata, CodiceCorsa
Persone	Individui che partecipano.	CF, Cognome, Nome, DataDiNascita	CF
Prenotazione	Entità che consente all'acquirente di ottenere l'accesso alla corsa specifica sulla tratta selezionata.	CodiceP, PostoTreno, Tratta	CodiceP, DataCorsa, CodiceTratta, CodiceCorsa
Posto	Posto a sedere.	PostoTreno	PostoTreno, Vagone, Treno
Tratta	Percorso seguito dal Treno durante una Corsa.	CodiceTratta, CapolineaArrivo, CapolineaPartenza.	CodiceTrata
Treni	Mezzo di trasporto pubblico.	MatricolaTreno, DataDiAcquisti, StoricoManutenzioni, NumeroVagoni, NumeroVagoniPrima Classe, NumeroVagoniSecon daClasse,	MatricolaTreno
Vagone	Struttura su ruote	Marca, Modello, MaxPasseggeri	CodiceVagone, MatricolaTreno

4. Progettazione logica

Volume dei dati

Durante la fase di progettazione, si prevede di conservare i dati per un periodo massimo di dieci anni. Una volta trascorso questo lasso di tempo sarà possibile eliminare i dati obsoleti (prenotazioni e corse effettuate) per ottimizzare lo spazio di archiviazione e le prestazioni del sistema.

1. Vengono gestiti circa 80 treni che vengono revisionati su base settimanale. Il volume atteso nelle storiche manutenzione è di 41.600
2. Si prevede che il numero totale delle tratte gestite è 50.
3. Si ipotizza che un vagone abbia 20 posti e un treno ad alta velocità comprende 15 vagoni e 2 locomotive, dei 15 vagoni 10 sono di seconda classe e 5 di prima. Il volume atteso dei vagoni è 1200 e per le locomotive è 160.
4. Si assume che per ogni corsa il 70% delle disponibilità sia prenotato, quindi 210 prenotazioni per ogni corsa.
5. Di tutte le prenotazioni il 35% viene fatto da persone ricorrenti, il numero degli acquirenti è 36855000.
6. In media ogni tratta comprende 8 fermate e nel giro di 10 anni viene aggiunta una fermata all'anno, per un volume di 1210.
7. Vengono organizzati i turni di 130 lavoratori e ogni lavoratore lavora 5 giorni a settimana, si ha un totale di 385.714 turni gestiti dopo 10 anni. I lavoratori accedono per scaricare I turni una volta a settimana, si ha quindi un volume totale di 66.857 accessi.
8. Ogni Giorno tutte le tratte vengono coperte da almeno una corsa e il 50% delle tratte anche da 2 corse. Sono 75 corse al giorno e dopo 10 anni sono 270.000 corse fatte.
9. I treni sono 80, ogni treno ha 300 posti. I posti totali sono 24000
10. Ogni tratta ha 8 fermate in media e le corse che vengono fatte sono in tutto 270.000, gli orari salvati sono quindi 2.160.000.
11. In media ogni treno viene revisionato ogni mese.

Concetto nello schema	Tipo[1]	Volume atteso
Acquirente	E	36.855.000
Corsa	E	270.000
Fermate	E	410
Lavoratore	E	130
Locomotrici	E	160
Orari	E	2.160.000
Persona	E	36.855.130
Posto	E	24.000
Prenotazione	E	56.700.000
Tratta	E	50
Treni	E	80
Vagoni	E	1200
Appartiene	R	24.000
Capotreno	R	80
Collegata	R	56.700.000
Composto	R	1200
Comprende	R	56.700.000
Coperta da	R	270.000
Determina	R	2.160.000
Effettua	R	56.700.000
Formato da	R	160
Include	R	410
Lavora	R	270.000
Macchinista	R	80
Usa	R	270.000

[1] Indicare con E le entità, con R le relazioni

Tavola delle operazioni

Cod.	Descrizione	Frequenza attesa
AG1	Report turni di Lavoro	130/Settimana ≃ 19/Giorno
AG2	Registra Prenotazione	472.500/Mese ≃ 15.750/Giorno
AG3	Registra Manutenzione	80/Mese ≃ 3/Giorno
AG4	Registra Corse	2250/Mese ≃ 75/Giorno
AG5	Registra Treno	0,66/Mese
AG6	Registra Locomotrice	1,3/Mese
AG7	Registra Tratta	1,25/Mese
AG8	Registra Fermata	3,41/Mese
AG9	Registra Vagone	10/Mese
AG10	Visualizza Prenotazioni	472.500/Mese ≃ 15.750/Giorno
L2	Registrazione	307.126/Mese ≃ 10.237/Giorno
L1	Login	307.126/Mese ≃ 10.237/Giorno

Costo delle operazioni

Si suppone che il costo di scrittura di un dato sia il doppio del costo in lettura.

Si calcolano gli accessi in lettura e scrittura che vengono fatti sulle entità ogni volta che si effettua un'operazione.

Operazione L1-Login

Concetto	Costrutto	Accessi	Tipo
Persone	E	1	L

Il costo dell'operazione è di un accesso, dalla frequenza dell'operazione si ottengono 307.126 accessi al mese.

Operazione L2-Registrazione

Concetto	Costrutto	Accessi	Tipo
Persone	E	1	L

Il costo dell'operazione è di 2 accessi, considerando la frequenza dell'operazione otteniamo 614.252 accessi al mese.

Operazione AG1-Report Turni Di Lavoro

Concetto	Costrutto	Accessi	Tipo
Corsa	E	14	L
Orari	E	14	L

Il costo dell'operazione è di 28 accessi, si ipotizza che ogni lavoratore faccia 5 turni a Settimana e ogni volta vengono riportati i turni della settimana passata e la prossima a venire.
La frequenza dell'operazione è di 130 ogni settimana, si ottengono quindi 3640 accessi alla settimana e quindi 14560 accessi al mese.

Operazione AG2-Registra Prenotazione

Concetto	Costrutto	Accessi	Tipo
Include	E	1200	L
Orari	E	14	L
Corsa	E	1	L
Corsa	E	1	L
Vagone	E	15	L
Prenotazione	E	1	S

L'operazione richiede 1.216 accessi, poiché è necessario selezionare la tratta da assegnare, includendo le fermate associate secondo la relazione "include". Successivamente, vengono letti gli orari delle fermate collegate alla corsa, con una media di 8 fermate, per un totale di 14 orari. Inoltre, si procede alla lettura del codice della corsa, del treno assegnato e dei vagoni che lo compongono.
Considerando che questa operazione viene eseguita 472.500 volte al mese, il volume totale di accessi mensili ammonta a 582.592.500.

Operazione AG3-Registra Manutenzione

Concetto	Costrutto	Accessi	Tipo
Treno	E	80	L
Manutenzione	E	1	S

Operazione AG4-Registra Corsa

Concetto	Costrutto	Accessi	Tipo
Treno	E	80	L
Include	E	1200	L
Lavoratore	E	130	L
Corsa	E	3600	L
Orari	E	28.800	L
Orari	E	8	S
Corsa	E	1	S

L'operazione richiede 33.828 accessi, poiché è necessario esaminare tutti gli 80 treni disponibili per determinare a quale assegnare la corsa. Successivamente, bisogna selezionare la tratta da percorrere, includendo le fermate associate secondo la relazione "include", e individuare i lavoratori da assegnare alla corsa. È inoltre necessario verificare che né il treno né il lavoratore siano già impegnati in altre corse, registrare la nuova corsa e salvare gli orari delle fermate corrispondenti.
Dato che questa operazione viene eseguita 2.250 volte al mese, il volume totale di accessi mensili ammonta a 76.113.000.

Operazione AG5-Registra Treno

Concetto	Costrutto	Accessi	Tipo
Lavoratore	E	130	L
Treno	E	1	S

L'operazione richiede 132 accessi, poiché è necessario leggere tutti i 130 lavoratori disponibili prima di procedere alla registrazione del treno.
Dato che questa operazione avviene con una frequenza di 0,66 volte al mese, arrotondando a 1, si ottiene un totale di 132 accessi mensili.

Operazione AG6-Registra Locomotrice

Concetto	Costrutto	Accessi	Tipo
Treno	E	80	L
Locomotrice	E	1	S

L'operazione richiede un totale di 82 accessi, poiché è necessario prima leggere i dati di tutti i 80 treni disponibili e successivamente registrare la locomotrice, associandola al treno corrispondente. Dato che la frequenza di questa operazione è di 1,3 volte al mese, arrotondando per difetto a 1, si ottiene un volume complessivo di 82 accessi mensili.

Operazione AG7-Registra Tratta

Concetto	Costrutto	Accessi	Tipo
Fermata	E	410	L
Tratta	E	1	S
Include	E	8	S

L'operazione comporta un totale di 428 accessi, di cui 410 derivano dalla lettura di tutte le fermate esistenti prima della registrazione di una nuova tratta e 8 accessi in scrittura per aggiornare la relazione "include", considerando che ogni tratta comprende mediamente 8 fermate. Dato che questa operazione avviene con una frequenza di 1,25 volte al mese, arrotondando per difetto a 1, il volume totale di accessi mensili risulta 428.

Operazione AG8-Registra Fermata

Concetto	Costrutto	Accessi	Tipo
Fermata	E	1	S

L'operazione comporta un costo di 2 accessi. Dato che la frequenza dell'operazione è di 3,41 volte al mese, arrotondando per difetto a 3, si ottiene un totale di 6 accessi mensili.

Operazione AG9-Registra Vagone

Concetto	Costrutto	Accessi	Tipo
Treno	E	80	L
Vagone	E	1	S

L'operazione ha un costo di 82 accessi, poiché è necessario prima leggere tutti i 80 treni e successivamente registrare il vagone associato a uno di essi. Considerando una frequenza di 10 operazioni al mese, il totale degli accessi mensili ammonta a 820.

Operazione AG10-Visualizza Prenotazioni

Concetto	Costrutto	Accessi	Tipo
Prenotazione	E	1	L
Corsa	E	1	L
Orari	E	2	L

L'operazione ha un costo di 4 accessi. Data una frequenza di 15.750 operazioni al giorno, il totale degli accessi giornalieri è 63.000, portando a un totale mensile di 1.890.000 accessi.

Ristrutturazione dello schema E-R

Lo schema di ristrutturazione è suddivisibile in:

- Analisi Ridondanze.
- Eliminazione Generalizzazioni.
- Partizione/Accorpamento delle Entità e Associazioni.

Analisi Ridondanze

Dall'analisi dello schema E-R emergono alcune ridondanze. Ad esempio, l'attributo "NumeroVagoni" risulta superfluo poiché il suo valore può essere dedotto dall'associazione "Composto", così come gli attributi "NumeroVagoniPrimaClasse" "NumeroVagoniSecondaClasse". Allo stesso modo, l'attributo "Numero Treni" nell'entità "Tratta" può essere ricavato dall'entità "Corsa" tramite una query. Dato che il numero di vagoni viene determinato una sola volta per ogni treno, si è deciso di eliminare l'attributo "NumeroVagoni", insieme a "NumeroVagoniPrimaClasse" e "NumeroVagoniSecondaClasse". Sebbene questi tre attributi non rappresentino un costo elevato in termini di memoria (ad esempio, essendo di tipo INTEGER, la loro presenza implicherebbe un'occupazione di 4 byte * 80 treni * 3 attributi = 960 byte), la loro rimozione consente una gestione più efficiente dei dati. Per ovviare alla loro assenza, è stato introdotto un nuovo attributo "Classe" nell'entità "Vagoni", che permette di rappresentare le diverse tipologie di vagoni in modo più strutturato, riducendo la ridondanza.
Inoltre, nel caso in cui venga eliminato un vagone da un treno, non sarà necessario aggiornare manualmente gli attributi "Numero Vagoni", "NumeroVagoniPrimaClasse" e "NumeroVagoniSecondaClasse", evitando possibili incoerenze nei dati.

Lo stesso discorso si applica all'attributo "Numero Treni" dell'entità "Tratta", che viene rimosso per prevenire problemi di ridondanza e inconsistenza. Se, infatti, il numero di treni su una tratta dovesse variare, l'attributo "Numero Treni" andrebbe aggiornato manualmente, con il rischio di introdurre errori.

Un'altra ottimizzazione riguarda gli attributi "capolineaPartenza" e "capolineaArrivo" dell'entità "Tratta", che sono stati eliminati poiché le informazioni sui capolinea possono essere ottenute direttamente attraverso l'attributo "Numero Fermate". Una semplice query sulla relazione "Include" è sufficiente per individuare il capolinea di partenza e di arrivo per ciascuna tratta, evitando così possibili discrepanze nei dati.

Per quanto riguarda l'identificatore della "Corsa", è stato semplificato includendo solo "CodiceCorsa" e "Data", poiché questi due attributi garantiscono già un'unicità sufficiente. Di conseguenza, l'inclusione del "CodiceTratta" è stata considerata superflua. Un identificatore composto da tre colonne (id tratta, codice corsa, data) avrebbe richiesto indici più grandi e complessi, rallentando le operazioni di ricerca e aumentando i tempi di inserimento e aggiornamento dei dati.
Dal punto di vista delle prestazioni, un identificatore meno complesso migliora l'efficienza delle query, specialmente se l'attributo "id tratta" non è strettamente necessario per distinguere univocamente le corse.

Infine, è stato rimosso anche l'attributo "MaxPasseggeri" dall'entità "Vagone", poiché il numero massimo di passeggeri per ciascun vagone può essere ottenuto direttamente dalla tabella "Posto", evitando ridondanze e garantendo una maggiore coerenza nei dati. Analizzando il costo totale:
Dimensione di un attributo int: 4 byte
Numero totale di tuple: 1200
Dimensione totale di tutte le tuple: 1200x4byte = 4800bytes
Se eliminiamo questo attributo, nel momento in cui eseguiamo l'operazione Registra Prenotazione abbiamo:

Concetto	Costrutto	Accessi	Tipo
Include	E	1200	L
Orari	E	14	L
Corsa	E	1	L
Corsa	E	1	L
Vagone	E	15	L
Posto	E	375	L
Prenotazione	E	1	S

Il costo totale senza l'attributo ridondante aumenta a 1.607 accessi, poiché per determinare il numero di posti disponibili è necessario scansionare ogni vagone del treno, leggendo tupla per tupla. Considerando una frequenza di 472.500 operazioni mensili, questo si traduce in 759.307.500 accessi al mese, ovvero 176.715.000 accessi in più rispetto alla soluzione con l'attributo ridondante. Per questo motivo, al fine di ottimizzare le prestazioni, si è deciso di mantenere l'attributo MaxPasseggeri.

Eliminazione Generalizzazione

Un altro aspetto da considerare è la gestione dell'unica generalizzazione presente nel sistema. Per eliminarla, la strategia più efficiente consiste nell'accorpare l'entità padre direttamente nelle entità figlie. Questa scelta è giustificata dal fatto che l'entità "Persone" non partecipa ad alcuna associazione, mentre le entità "Acquirente" e "Lavoratore" sono già coinvolte in relazioni specifiche. Di conseguenza, non sarà necessario introdurre ulteriori associazioni nel modello.

L'unico svantaggio di questa soluzione è l'aumento del numero di attributi nelle entità "Acquirente" e "Lavoratore", poiché erediteranno tutte le proprietà dell'entità padre. Tuttavia, questo impatto è trascurabile, considerando che l'aggiunta di nuovi attributi comporta un costo minimo di 1 byte per attributo.

Un'alternativa meno efficiente sarebbe mantenere l'entità padre e accorpare le entità figlie al suo interno. In questo caso, ci troveremmo con una tabella "Persone" contenente 36.855.130 tuple, il che influenzerebbe significativamente le prestazioni del database, specialmente durante operazioni frequenti come registrazione treni, registrazione corse e altre transazioni. Per comprendere meglio le implicazioni di questa scelta, confrontiamo due scenari:

1. Efficienza degli indici:
    - Se la tabella contiene solo 130 tuple, il database può gestirle interamente in memoria, garantendo un accesso rapido.
    - Se invece la tabella ha 36.855.130 tuple, diventa essenziale avere indici ben strutturati per evitare scansioni complete, che sarebbero altamente inefficaci. Tuttavia, mantenere e aggiornare indici su tabelle di grandi dimensioni comporta un costo computazionale maggiore.
2. Utilizzo della cache:
    - Una tabella con 130 tuple può essere interamente caricata in cache, velocizzando notevolmente le operazioni di lettura.
    - Una tabella molto più grande non può essere completamente mantenuta in memoria, portando a frequenti accessi al disco, che risultano più lenti rispetto all'accesso alla RAM.
3. Tempo di ricerca:
    - Con una tabella piccola, il tempo di ricerca è minimo, in quanto l'intero dataset può essere rapidamente scansionato.

- In una tabella più grande, la velocità di ricerca dipende dalla qualità degli indici. Se questi non sono ben ottimizzati, individuare 130 righe specifiche richiederebbe una scansione molto più onerosa in termini di tempo.

4. Contesa delle risorse:
    - Tabelle piccole generano un carico minore su CPU, memoria e I/O del disco, garantendo un funzionamento più fluido del sistema.
    - Tabelle molto grandi aumentano la competizione per le risorse, specialmente in presenza di numerose operazioni concorrenti di lettura e scrittura, con un conseguente degrado delle prestazioni complessive.

Sebbene l'operazione di lettura delle 130 righe sia la stessa in entrambi i casi, le prestazioni complessive del database variano significativamente a seconda della struttura scelta. L'accorpamento delle entità figlie direttamente nelle rispettive tabelle risulta la soluzione migliore, poiché evita accessi ridondanti, ottimizza la gestione della memoria e riduce il carico sulle risorse del database.

Partizionamento/Accorpamento delle Entità e Associazioni

L'ultima modifica apportata riguarda la separazione dell'attributo multivalore "storico manutenzioni" in un'entità dedicata, denominata "Storico Manutenzioni". Questa scelta consente di eliminare le ridondanze che si sarebbero generate mantenendo l'attributo all'interno della stessa entità.

In fase di modellazione concettuale, la tabella "Prenotazione" era inizialmente progettata con una chiave primaria composta da tre attributi:

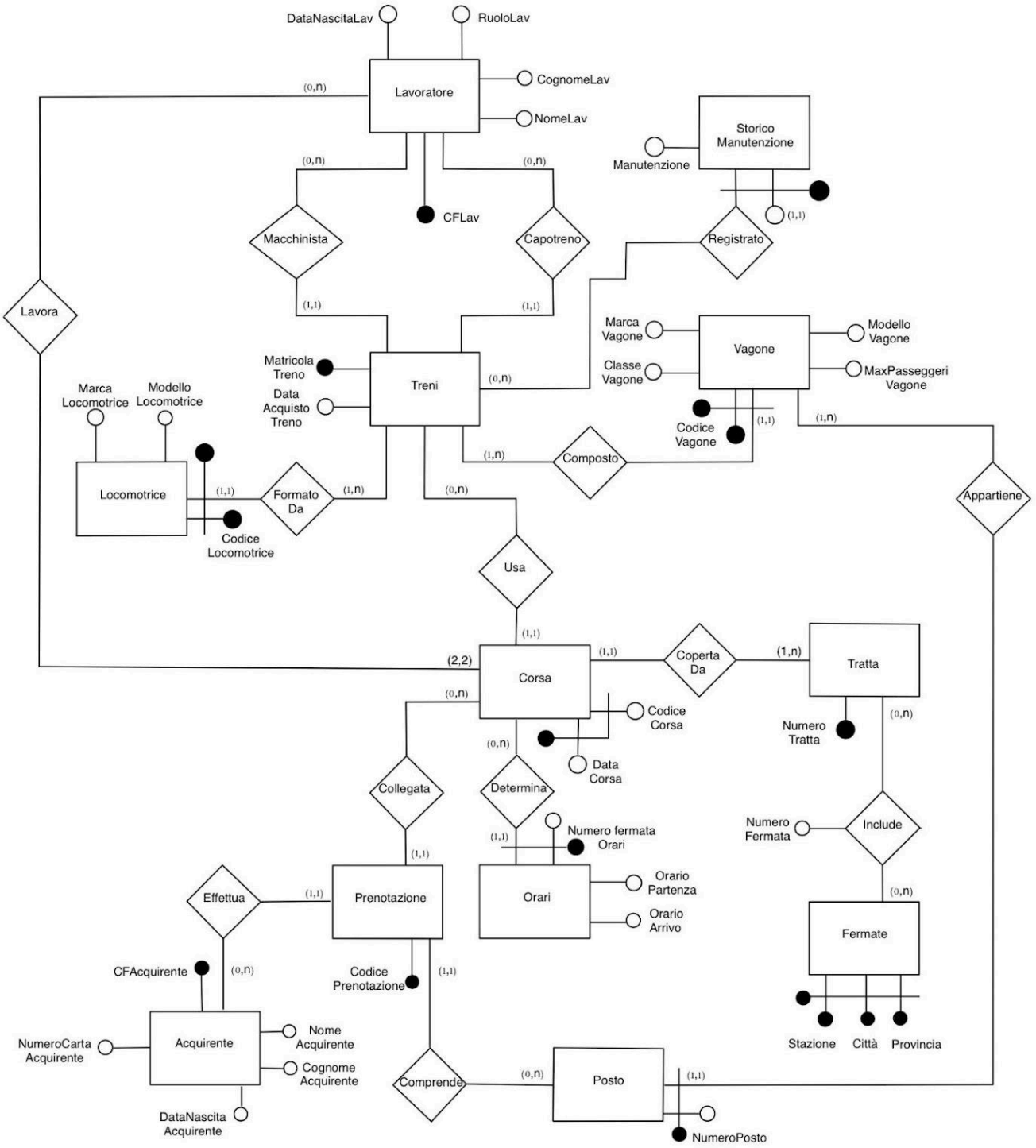
- codice_prenotazione: identificatore univoco di ogni prenotazione.
- codice_corsa: identificativo della corsa associata.
- data_corsa: data della corsa specifica.

Questa struttura era stata scelta per garantire l'univocità delle prenotazioni in relazione a una determinata corsa e data.

Un'analisi più approfondita delle esigenze del sistema ha portato a considerazioni importanti:

1. Univocità e Identificazione
    - È stato verificato che codice_prenotazione è sufficiente per identificare in modo univoco ogni prenotazione.
    - Poiché ogni codice è generato in modo unico, l'aggiunta di codice_corsa e data_corsa nella chiave primaria risulta superflua.
2. Miglioramento dell'Integrità Referenziale
    - Utilizzare una chiave primaria più semplice, basata solo su codice_prenotazione, facilita la gestione del database e migliora le prestazioni delle operazioni CRUD (Create, Read, Update, Delete).
    - La riduzione della complessità della chiave primaria rende le operazioni di ricerca e aggiornamento più efficienti, riducendo il rischio di errori e migliorando la scalabilità del sistema.

L'adozione di una chiave primaria più semplice e la separazione dello storico manutenzioni in un'entità dedicata permettono di ottimizzare l'architettura del database, migliorando sia la gestione dei dati sia l'efficienza delle operazioni.


DataNascitaLav
RuoloLav
Lavoratore
CognomeLav
NomeLav
(0,n)
CFLav
Macchinista
Capotreno
(1,1)
Lavora
Storico Manutenzione
Manutenzione
Registrato
Marca Vagone
Classe Vagone
Vagone
Modello Vagone
MaxPasseggeri Vagone
Codice Vagone
Matricola Treno
Data Acquisto Treno
Treni
Marca Locomotrice
Modello Locomotrice
Locomotrice
Codice Locomotrice
Formato Da
Composto
(1,n)
Appartiene
Usa
(2,2)
Corsa
Codice Corsa
Data Corsa
Coperta Da
Tratta
Numero Tratta
Collegata
Determina
Numero fermata
Orari
Numero Fermata
Include
Prenotazione
Effettua
Orario Partenza
Orario Arrivo
Fermate
Stazione
Città
Provincia
CFAcquirente
Codice Prenotazione
Acquirente
NumeroCarta Acquirente
Nome Acquirente
Cognome Acquirente
DataNascita Acquirente
Comprende
Posto
NumeroPosto

Trasformazione di attributi e identificatori

- ***matricolaTreno*** è Foreign Key associata alla '***Matricola Treno***' dell'entità ***Treno***
- ***idTratta*** è Foreign Key associata al '***Numero Tratta***' dell'entità ***Tratta***.
- ***CFMacchinista***, ***CFCapotreno*** sono Foreign Key associate al '***CFLav***' dell'entità ***Lavoratore***.

Traduzione di entità e associazioni

- Treno(***matricolaTreno***, dataDiAcquisto, CFMacchinista, CFCapotreno)
    - Treno(CFMacchinista) ⊆ Lavoratore(CFLav)
    - Treno(CFCapotreno) ⊆ Lavoratore(CFLav)
- StoricoManutenzione(***dataManutenzione, matricolaTreno***, manutenzione)
    - StoricoManutenzione(matricolaTreno) ⊆ Treno(matricolaTreno)
- Vagone(***codiceVagone,matricolaTreno***,modelloVagone,marcaVagone,maxPasseggeriVagone, classeVagone)
    - Vagone(matricolaTreno) ⊆ Treno(matricolaTreno)
- Locomotrice(***codiceLocomotrice, matricolaTreno***, modelloLocomotrice, marcaLocomotrice)
    - Locomotrice(matricolaTreno) ⊆ Treno(matricolaTreno)
- Tratta(***numeroTratta***)
- Include(***idTratta, stazione, città, provincia***, numeroFermata)
    - Include(stazione, città, provincia) ⊆ Fermate(stazione, città, provincia)
    - Include(idTratta) ⊆ Tratta(numeroTratta)
- Fermate(***stazione, città, provincia***)
- Prenotazione(***codicePrenotazione***,codiceCorsa,dataCorsa,numeroPostoTreno,numeroVagone, matricolaTreno, CFAcquirente)
    - Prenotazione(CFAcquirente) ⊆ Acquirente(CF)
    - Prenotazione(codiceCorsa) ⊆ Corsa(idCorsa)
    - Prenotazione(dataCorsa) ⊆ Corsa(dataCorsa)
    - Prenotazione(numeroPostoTreno) ⊆ Posto(postoTreno)
    - Prenotazione(numeroVagone) ⊆ Posto(numeroVagone)
    - Prenotazione(matricolaTreno) ⊆ Posto(matricolaTreno)
- Lavoratore(***CFLav***, nomeLav, cognomeLav, dataDiNascitaLav, ruoloLav)
- Acquirente(***CFAcquirente***, nomeAcquirente, cognomeAcquirente, dataDiNascitaAcquirente, numeroCartaAcquirente)

- Orari(***numeroFermataOrari, codiceCorsa, dataCorsa***, orarioPartenza, orarioArrivo)
    - Orari(codiceCorsa) ⊆ Corsa(idCorsa)
    - Orari(dataCorsa) ⊆ Corsa(dataCorsa)
- Corsa(***idCorsa, dataCorsa***, idTratta, matricolaTreno, CFLav1, CFLav2)
    - Corsa(idCorsa) ⊆ Tratta(numeroTratta)
    - Corsa(matricolaTreno) ⊆ Treno(matricolaTreno)
    - Corsa(CFLav1) ⊆ Lavoratore(CFLav)
    - Corsa(CFLav2) ⊆ Lavoratore(CFLav)
- Posto(***postoTreno, numeroVagone, matricolaTreno***)
    - Posto(numeroVagone) ⊆ Vagone(codiceVagone)
    - Posto(matricolaTreno) ⊆ Vagone(matricolaTreno)

Normalizzazione del modello relazionale

1NF - Prima Forma Normale

Una relazione è in 1NF quando:

- E' presente una chiave primaria
- Non vi sono attributi ripetuti
- Gli attributi non sono costituiti da strutture complesse

Le tabelle analizzate rispettano queste condizioni: ogni colonna contiene un solo valore per riga, non si riscontrano insiemi di attributi ripetuti o gruppi di colonne e le chiavi esterne sono utilizzate correttamente per collegare le varie tabelle.

2NF - Seconda Forma Normale

Una relazione soddisfa la 2NF quando:

- Ogni chiave è composta dal numero minimo di attributi necessario per identificarla univocamente, non include quindi sottoinsiemi che costituiscano a loro volta chiavi.

Nelle tabelle analizzate tutte le chiavi primarie risultano minimali.

3NF - Terza Forma Normale

Una relazione soddisfa la 3NF quando:

- Per ogni dipendenza funzionale del tipo $X \rightarrow A$, è verificata almeno una delle seguenti condizioni:
    - X è superchiave
    - A è un attributo appartenente ad una chiave candidata

Nelle tabelle analizzate, tutte le dipendenze rispettano uno dei due criteri sopra indicati.

5. Progettazione fisica

Utenti e privilegi

Nel sistema sono previsti vari ruoli con accesso a diverse operazioni, ognuno con un livello di accesso specifico:

- Login: Ha accesso alla L1 (Login)
- Acquirenti: Hanno accesso alle operazioni AG2 (Registra Prenotazione) e AG10 (Visualizza Prenotazione)
- Gestori del Servizio: Hanno accesso alle operazioni AG4, AG5, AG6, AG7, AG8, AG9 (tutte operazioni relative a gestione dei treni, fermate e tratte)
- Lavoratori: Hanno accesso all'operazione AG1 (Report turni di lavoro)
- Manutentore: Ha accesso all'operazione AG3 (Registra Manutenzione)

Strutture di memorizzazione

Tabella <Acquirente>		
Colonna	**Tipo di dato**	**Attributi[2]**
CF	VARCHAR(16)	PK, NN
cognome	VARCHAR(20)	NN
nome	VARCHAR(20)	NN
dataDiNascita	DATE	NN
username	VARCHAR(45)	NN, UNIQUE
password	VARCHAR(45)	NN
numeroCartaCredito	BIGINT	NN

Tabella <Corsa>		
Colonna	**Tipo di dato**	**Attributi[3]**
idCorsa	INT	PK, NN
dataCorsa	DATE	PK, NN
idTratta	INT	NN
matricolaTreno	VARCHAR(4)	NN
CFLavavoratore1	VARCHAR(16)	NN
CFLavoratore2	VARCHAR(16)	NN

[2] PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

[3] PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

Tabella <Fermate>		
Colonna	**Tipo di dato**	**Attributi**[4]
stazione	VARCHAR(50)	PK, NN
città	VARCHAR(50)	PK, NN
provincia	VARCHAR(50)	PK, NN

Tabella <Gestore del Servizio>		
Colonna	**Tipo di dato**	**Attributi**[5]
username	VARCHAR(20)	PK, NN
password	VARCHAR(45)	NN

Tabella <Include>		
Colonna	**Tipo di dato**	**Attributi**[6]
idTratta	INT	PK, NN
stazione	VARCHAR(50)	PK, NN
città	VARCHAR(50)	PK, NN
provincia	VARCHAR(50)	PK, NN
numeroFermata	INT	NN

Tabella <Lavoratore>		
Colonna	**Tipo di dato**	**Attributi**[7]
CF	VARCHAR(16)	PK, NN
nome	VARCHAR(20)	NN
cognome	VARCHAR(20)	NN
dataNascita	DATE	NN
username	VARCHAR(45)	NN, UNIQUE
password	VARCHAR(45)	NN
ruolo	VARCHAR(45)	NN

[4] PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

[5] PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

[6] PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

[7] PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

Tabella <Locomotrici>		
Colonna	**Tipo di dato**	**Attributi**[8]
codice	VARCHAR(10)	PK, NN
marca	VARCHAR(20)	NN
modello	VARCHAR(20)	NN
matricolaTreno	VARCHAR(4)	NN

Tabella <Manutentore>		
Colonna	**Tipo di dato**	**Attributi**[9]
username	VARCHAR(20)	PK, NN
password	VARCHAR(45)	NN

Tabella <Orari>		
Colonna	**Tipo di dato**	**Attributi**[10]
numeroFermataOrari	INT	PK, NN
orarioPartenza	TIME	
orarioArrivo	TIME	
idCorsa	INT	PK, NN
dataCorsa	DATE	PK, NN

Tabella <Posto>		
Colonna	**Tipo di dato**	**Attributi**[11]
id_vagone	VARCHAR(10)	PK, NN
posto_treno	VARCHAR(4)	PK, NN
matricolaTreno	INT	PK, NN

[8] PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

[9] PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

[10] PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

[11] PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

Tabella <Prenotazione>		
Colonna	**Tipo di dato**	**Attributi**[12]
codicePrenotazione	VARCHAR(20)	PK, NN
id_corsa	INT	NN
data	DATA	N
postoTreno	INT	NN
CFAcquirente	VARCHAR(16)	NN
id_vagone	VARCHAR(10)	NN
matricolaTreno	VARCHAR(4)	NN

Tabella <StoricoManutenzioni>		
Colonna	**Tipo di dato**	**Attributi**[13]
dataManutenzione	DATE	PK, NN
matricolaTreno	VARCHAR(4)	PK, NN
manutenzione	VARCHAR(100)	NN

Tabella <Tratta>		
Colonna	**Tipo di dato**	**Attributi**[14]
id	INT	PK,NN

Tabella <Treno>		
Colonna	**Tipo di dato**	**Attributi**[15]
matricola	VARCHAR(4)	PK, NN
dataAcquisto	DATE	NN
CFMacchinista	VARCHAR(16)	NN
CFCapotreno	VARCHAR(16)	NN

[12] PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

[13] PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

[14] PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

[15] PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

Tabella <Vagone>		
Colonna	**Tipo di dato**	**Attributi**[16]
codice	VARCHAR(10)	PK, NN
marca	VARCHAR(20)	NN
modello	VARCHAR(20)	NN
classe	INT	NN
maxPasseggeri	INT	NN
matricolaTreno	VARCHAR(4)	NN

Indici

Tabella <nome>	
Indice <nome>	**Tipo**[17]**:**

[16] PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

[17] IDX = index, UQ = unique, FT = full text, PR = primary.

Trigger

I quattro trigger sono stati creati per assicurarsi che l'username fornito in fase di inserimento o aggiornamento non compaia già in alcuna delle quattro tabelle indicate (GestoreDelServizio, Lavoratore, Manutentore, Acquirente). Questo meccanismo serve a mantenere l'unicità degli username, evitando che lo stesso venga utilizzato da utenti appartenenti a categorie diverse all'interno del sistema.

- **Trigger per il Gestore Di Servizi**

```
DELIMITER ;;
CREATE TRIGGER `before_insert_or_update_trigger_gestore` BEFORE INSERT ON `gestoredelservizio` FOR EACH ROW BEGIN
  DECLARE user_count INT;

  -- Verifica che il nome utente non sia già presente in nessuna delle tabelle
  SELECT COUNT(*) INTO user_count
  FROM (
    SELECT username FROM gestoredelservizio WHERE username = NEW.username
    UNION ALL
    SELECT username FROM lavoratore WHERE username = NEW.username
    UNION ALL
    SELECT username FROM manutentore WHERE username = NEW.username
    UNION ALL
    SELECT username FROM acquirente WHERE username = NEW.username
  ) AS user_union;

  IF user_count > 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Il nome utente è già in uso in una delle tabelle';
  END IF;
END */;;
DELIMITER ;
```

- **Trigger per il Lavoratore**

```
DELIMITER ;;
CREATE TRIGGER `before_insert_or_update_trigger_lavoratore` BEFORE INSERT ON `lavoratore` FOR EACH ROW BEGIN
  DECLARE user_count INT;

  -- Verifica che il nome utente non sia già presente in nessuna delle tabelle
  SELECT COUNT(*) INTO user_count
  FROM (
    SELECT username FROM gestoredelservizio WHERE username = NEW.username
    UNION ALL
    SELECT username FROM lavoratore WHERE username = NEW.username
    UNION ALL
    SELECT username FROM manutentore WHERE username = NEW.username
    UNION ALL
    SELECT username FROM acquirente WHERE username = NEW.username
  ) AS user_union;

  IF user_count > 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Il nome utente è già in uso in una delle tabelle';
  END IF;
END */;;
DELIMITER ;
```

- **Trigger per il Manutentore**

```
DELIMITER ;;
CREATE TRIGGER `before_insert_or_update_trigger_manutentore` BEFORE INSERT ON `manutentore` FOR EACH ROW BEGIN
  DECLARE user_count INT;

  -- Verifica che il nome utente non sia già presente in nessuna delle tabelle
  SELECT COUNT(*) INTO user_count
  FROM (
    SELECT username FROM gestoredelservizio WHERE username = NEW.username
    UNION ALL
    SELECT username FROM lavoratore WHERE username = NEW.username
    UNION ALL
    SELECT username FROM manutentore WHERE username = NEW.username
    UNION ALL
    SELECT username FROM acquirente WHERE username = NEW.username
  ) AS user_union;

  IF user_count > 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Il nome utente è già in uso in una delle tabelle';
  END IF;
END */;;
DELIMITER ;
```

- **Trigger per l'Acquirente**

```
DELIMITER ;;
CREATE TRIGGER `before_insert_or_update_trigger_acquirente` BEFORE INSERT ON `acquirente` FOR EACH ROW BEGIN
  DECLARE user_count INT;

  -- Verifica che il nome utente non sia già presente in nessuna delle tabelle
  SELECT COUNT(*) INTO user_count
  FROM (
    SELECT username FROM gestoredelservizio WHERE username = NEW.username
    UNION ALL
    SELECT username FROM lavoratore WHERE username = NEW.username
    UNION ALL
    SELECT username FROM manutentore WHERE username = NEW.username
    UNION ALL
    SELECT username FROM acquirente WHERE username = NEW.username
  ) AS user_union;

  IF user_count > 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Il nome utente è già in uso in una delle tabelle';
  END IF;
END */;;
DELIMITER ;
```

Eventi

È stato introdotto un evento periodico incaricato di eliminare i dati archiviati da oltre dieci anni, considerati non più significativi per il funzionamento dell'applicazione. Questo meccanismo consente di evitare un accumulo eccessivo di informazioni superflue all'interno del sistema, mantenendo il database leggero e ottimizzato. L'evento sarà eseguito per la prima volta al compimento dei dieci anni e si ripeterà con cadenza annuale, così da garantire nel tempo un volume di dati costante e controllato.

```
DELIMITER ;;
CREATE EVENT `eliminazione_dati_vecchi` ON SCHEDULE EVERY 1 YEAR STARTS '2024-06-23 11:29:41' ON COMPLETION PRESERVE ENABLE COMMENT 'Rimozione dati più vecchi di 10 anni - Test' DO BEGIN
   DELETE FROM orari
   WHERE dataCorsa <= DATE_SUB(NOW(), INTERVAL 10 YEAR);

   DELETE FROM prenotazione
   WHERE data_ <= DATE_SUB(NOW(), INTERVAL 10 YEAR);

   DELETE FROM corsa
   WHERE dataCorsa <= DATE_SUB(NOW(), INTERVAL 10 YEAR);
END */ ;;
```

Viste

Non utilizzate.

Funzioni

controllo_carta_di_credito

```
CREATE FUNCTION controllo_carta_di_credito(carta_di_credito VARCHAR(16)) RETURNS TINYINT(1)
   DETERMINISTIC
BEGIN
   -- Gestione del caso NULL
   IF carta_di_credito IS NULL THEN
      RETURN TRUE; -- oppure FALSE, se vuoi impedirlo
   END IF;

   -- Regex: deve contenere esattamente 16 cifre
   IF carta_di_credito REGEXP '^[0-9]{16}$' THEN
      RETURN TRUE;
   ELSE
      RETURN FALSE;
   END IF;
END;;
```

controllo_cf

Controlla che il codice fiscale inserito rispetti il formato alfanumerico italiano standard.

```
CREATE FUNCTION controllo_cf(codice_fiscale VARCHAR(16)) RETURNS TINYINT(1)
   DETERMINISTIC
BEGIN
   IF codice_fiscale IS NULL THEN
      RETURN TRUE; -- Considera NULL come accettabile (facoltativo)
   END IF;

   IF codice_fiscale REGEXP '^[A-Z]{6}[0-9]{2}[A-Z][0-9]{2}[A-Z][0-9]{3}[A-Z]$' THEN
      RETURN TRUE;
   ELSE
      RETURN FALSE;
   END IF;
END;;
```

controlla_classe

Verifica che la classe di un vagone sia valida, cioè pari a 1 o 2.

```
CREATE DEFINER=`root`@`localhost` FUNCTION `controlla_classe`(classe int) RETURNS tinyint(1)
   DETERMINISTIC
BEGIN
        if classe =1 OR classe=2 then
                return true;
        else
                return false;
        end if;
END ;;
```

controllo_lavoratore

Verifica che il lavoratore selezionato non sia impegnato in altre corse durante la fascia oraria specificata.

```
CREATE DEFINER=`root`@`localhost` FUNCTION `controllo_lavoratore`(
   CFLavoratore1_ VARCHAR(16),
   CFLavoratore2_ VARCHAR(16),
   var_data DATE,
   orario_partenza TIME,
   orario_arrivo TIME
) RETURNS tinyint(1)
   DETERMINISTIC
BEGIN
   DECLARE sovrapposizione BOOLEAN;

   SELECT
      EXISTS (
         SELECT 1
         FROM (
            SELECT
               orarioPartenza,
               orarioArrivo
            FROM
               (SELECT
                  corsa.idCorsa,
                  corsa.dataCorsa,
                  corsa.matricolaTreno,
                        CASE WHEN orari.orarioArrivo = '00:00:00' THEN orari.orarioPartenza ELSE
orari.orarioArrivo END AS orarioPartenza,
                        CASE WHEN orari.orarioPartenza = '00:00:00' THEN orari.orarioArrivo ELSE
orari.orarioPartenza END AS orarioArrivo
               FROM
                  orari
               JOIN
                  corsa ON corsa.dataCorsa = orari.dataCorsa AND corsa.idCorsa = orari.idCorsa
```

```
            WHERE
                        (corsa.CFLavoratore1 = CFLavoratore1_ OR corsa.CFLavoratore2 =
CFLavoratore2_)
              AND corsa.dataCorsa = var_data
            ) AS combined_results
        ) AS subquery
        WHERE
          (orario_partenza BETWEEN orarioPartenza AND orarioArrivo
          OR orario_arrivo BETWEEN orarioPartenza AND orarioArrivo
          OR orarioPartenza BETWEEN orario_partenza AND orario_arrivo
          OR orarioArrivo BETWEEN orario_partenza AND orario_arrivo)
      ) INTO sovrapposizione;

   RETURN sovrapposizione;
END ;;
```

controllo_treno

Verifica che il treno indicato sia libero nella data e nella fascia oraria specificata.

```
CREATE DEFINER=`root`@`localhost` FUNCTION `controllo_treno`(
   treno VARCHAR(16),
   var_data DATE,
   orario_partenza TIME,
   orario_arrivo TIME
) RETURNS tinyint(1)
   DETERMINISTIC
BEGIN
   DECLARE sovrapposizione BOOLEAN;

   SELECT
      EXISTS (
         SELECT 1
         FROM (
            SELECT
               orarioPartenza,
               orarioArrivo
            FROM
               (SELECT
                  corsa.idCorsa,
                  corsa.dataCorsa,
                  corsa.matricolaTreno,
                        CASE WHEN orari.orarioArrivo = '00:00:00' THEN orari.orarioPartenza ELSE
orari.orarioArrivo END AS orarioPartenza,
                        CASE WHEN orari.orarioPartenza = '00:00:00' THEN orari.orarioArrivo ELSE
orari.orarioPartenza END AS orarioArrivo
               FROM
                  orari
               JOIN
                  corsa ON corsa.dataCorsa = orari.dataCorsa AND corsa.idCorsa = orari.idCorsa
               WHERE
                  corsa.matricolaTreno = treno AND corsa.dataCorsa = var_data
```

```
            ) AS combined_results
        ) AS subquery
        WHERE
          (orario_partenza BETWEEN orarioPartenza AND orarioArrivo
          OR orario_arrivo BETWEEN orarioPartenza AND orarioArrivo
          OR orarioPartenza BETWEEN orario_partenza AND orario_arrivo
          OR orarioArrivo BETWEEN orario_partenza AND orario_arrivo)
      ) INTO sovrapposizione;

  RETURN sovrapposizione;
END ;;
```

controllo_matricola

Controlla che la matricola del treno sia lunga esattamente 4 caratteri.

```
CREATE DEFINER=`root`@`localhost` FUNCTION `controllo_matricola`(matricola
VARCHAR(4)) RETURNS tinyint(1)
  DETERMINISTIC
BEGIN
  IF LENGTH(matricola) = 4 THEN
    RETURN 1; -- Restituisce vero (1) se la lunghezza è 4
  ELSE
    RETURN 0; -- Restituisce falso (0) se la lunghezza non è 4
  END IF;
END ;;
```

max_passeggeri

Controlla che un vagone non abbia più di 25 passeggeri come capacità massima.

```
CREATE DEFINER=`root`@`localhost` FUNCTION `max_passeggeri`(max_passeggeri INT)
RETURNS tinyint(1)
   DETERMINISTIC
BEGIN
        if max_passeggeri <= 25 then
                return true;
        else
                return false;
        end if;
END ;;
```

controllo_orari

Controlla che l'orario di partenza preceda quello di arrivo.

```
CREATE DEFINER=`root`@`localhost` FUNCTION `controllo_orari`(orario_partenza Time,
oraraio_arrivo time) RETURNS tinyint(1)
   DETERMINISTIC
BEGIN
        if orario_partenza > oraraio_arrivo THEN
                return true;
        else
                return false;
        end if;
END ;;
```

controlla_lavoratori_treno

```
CREATE  DEFINER=`root`@`localhost`  FUNCTION  `controlla_lavoratori_treno`(var_macchinista
varchar(16),var_capotreno varchar(16),matricolaTreno varchar(4)) RETURNS tinyint(1)
   DETERMINISTIC
BEGIN

        DECLARE macchinista VARCHAR(16);
   DECLARE capotreno VARCHAR(16);

        SELECT CFMacchinista,CFCapotreno
   INTO macchinista,capotreno
   from treno
   WHERE matricola=matricolaTreno;



        if var_macchinista=macchinista AND var_capotreno=capotreno  then
                return true;
        else
                return false;
        end if;
END ;;
```

Stored Procedures e transazioni

Procedura registra_treno

Questa procedura ha il compito di inserire un nuovo treno nel sistema. Poiché all'interno della transazione non vengono effettuate operazioni di lettura su dati concorrenti, non si rischiano anomalie di isolamento, pertanto viene utilizzato il livello di isolamento `READ UNCOMMITTED`.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `registra_treno`(
        in var_matricola varchar(4),
   in var_dataDiAcquisto varchar(45),
   in var_macchinista varchar(45),
   in var_capotreno varchar(45)
)
BEGIN
        declare exit handler for sqlexception
   begin
                rollback;
      resignal;
        end;

        set transaction isolation level READ uncommitted;
        start transaction;

        if(`DB_ferroviario`.controllo_matricola(var_matricola) is false) then
                signal sqlstate '45011' set message_text = 'Inserire il codice del treno di 4 caratteri.';
        end if;
        insert into treno values (var_matricola, var_dataDiAcquisto,var_macchinista,var_capotreno);

   commit;

END ;;
```

Procedura registra_vagone

Questa procedura serve ad aggiungere un nuovo vagone al database. Non essendoci operazioni di lettura concorrenti durante la transazione, non si verificano problemi di isolamento. Per questo motivo viene utilizzato il livello `READ UNCOMMITTED`.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `registra_vagone`(
   IN var_id VARCHAR(10),
   IN var_marca VARCHAR(45),
   IN var_classe INT,
   IN var_modello VARCHAR(45),
   IN var_numMaxPasseggeri INT,
   IN var_matricolaTreno VARCHAR(4)
)
BEGIN
   DECLARE EXIT HANDLER FOR SQLEXCEPTION
   BEGIN
      ROLLBACK;
      RESIGNAL;
   END;

   SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
   START TRANSACTION;

   -- Controllo numero massimo passeggeri
   IF (`DB_ferroviario`.max_passeggeri(var_numMaxPasseggeri) IS FALSE) THEN
       SIGNAL SQLSTATE '45008' SET MESSAGE_TEXT = 'Il numero di passeggeri è maggiore di
25.';
   END IF;

   -- Controllo validità classe
   IF (`DB_ferroviario`.controlla_classe(var_classe) IS FALSE) THEN
      SIGNAL SQLSTATE '45013' SET MESSAGE_TEXT = 'Il vagone può essere di 1ª o 2ª classe.';
   END IF;
```

```
   -- Inserimento nel DB
   INSERT INTO vagone (codice, marca, classe, modello, maxPasseggeri, matricolaTreno)
            VALUES (var_id, var_marca, var_classe, var_modello, var_numMaxPasseggeri,
var_matricolaTreno);

   COMMIT;
END ;;
```

Procedura registra_Fermate

Questa procedura consente l'inserimento di una nuova fermata all'interno del sistema. Dato che la transazione non prevede operazioni di lettura, non sussistono problematiche legate all'isolamento. Per questo motivo si adotta il livello `READ UNCOMMITTED`.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `registra_Fermate`(
   IN stazione VARCHAR(50),
   IN citta VARCHAR(50),
   IN provincia VARCHAR(50)
)
BEGIN



   DECLARE exit handler for sqlexception
   BEGIN
      ROLLBACK;
      RESIGNAL;
   END;



   SET TRANSACTION ISOLATION LEVEL READ uncommitted;
   START TRANSACTION;

   INSERT INTO fermate VALUES(stazione, citta, provincia);

   COMMIT;
END ;;
DELIMITER ;
DELIMITER ;;
CREATE DEFINER=`root`@`localhost` PROCEDURE `registra_locomotrici`(
         IN vard_id INT,
   IN var_marca VARCHAR(20),
   IN var_modello VARCHAR(20),
```

```
   IN var_matricolaTreno INT)
BEGIN
        declare exit handler for sqlexception
        begin
                rollback;
      resignal;
        end;

   set transaction isolation level Read Uncommitted ;
        start transaction;

                insert into `locomotrici` (`codice`, `marca`, `modello`, `matricolaTreno`)
                        values (vard_id, var_marca, var_modello, var_matricolaTreno);
        commit;

END ;;
```

Procedura registra_locomotrici

La seguente procedura consente di aggiungere una nuova locomotrice nel database. Considerando che all'interno della transazione non vengono eseguite letture su dati condivisi, non si incorre in problematiche di concorrenza, pertanto si imposta il livello di isolamento su `READ UNCOMMITTED`.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `registra_locomotrici`(
        IN vard_id INT,
   IN var_marca VARCHAR(20),
   IN var_modello VARCHAR(20),
   IN var_matricolaTreno INT)
BEGIN
        declare exit handler for sqlexception
        begin
                rollback;
      resignal;
        end;

   set transaction isolation level Read Uncommitted ;
        start transaction;

                insert into `locomotrici` (`codice`, `marca`, `modello`, `matricolaTreno`)
                        values (vard_id, var_marca, var_modello, var_matricolaTreno);
        commit;

END ;;
```

Procedura registra_Tratta

Questa procedura consente di creare una nuova tratta ferroviaria. Le fermate che la compongono vengono passate come stringa text, in cui ogni stazione è separata da una virgola. Il ciclo elabora_fermate estrae una fermata alla volta dalla stringa ed effettua l'inserimento nella tabella di associazione. Considerando il basso tasso di concorrenza su questa procedura, viene scelto il livello di isolamento `REPEATABLE READ`, che garantisce consistenza stabile dei dati letti per l'intera durata della transazione.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `registra_Tratta`(
   IN fermate TEXT
)
BEGIN
   DECLARE citta_ VARCHAR(50);
   DECLARE provincia_ VARCHAR(50);
   DECLARE current_fermata VARCHAR(50);
   DECLARE start INT DEFAULT 1;
   DECLARE length INT DEFAULT 0;
   DECLARE done INT DEFAULT 0;
   DECLARE max_id INT DEFAULT 1;
   DECLARE counter INT DEFAULT 1;
   DECLARE err_msg TEXT;

   DECLARE EXIT HANDLER FOR SQLEXCEPTION
   BEGIN
      ROLLBACK;
      RESIGNAL;
   END;

   SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
   START TRANSACTION;

   SELECT MAX(id) INTO max_id FROM tratta;
   SET max_id = IFNULL(max_id, 0) + 1;
```

```
INSERT INTO tratta(id) VALUES(max_id);

cursor_loop: LOOP
    SET length = LOCATE(',', fermate, start) - start;

    IF length < 0 THEN
        SET current_fermata = TRIM(SUBSTRING(fermate, start));
        SET done = 1;
    ELSE
        SET current_fermata = TRIM(SUBSTRING(fermate, start, length));
    END IF;

    IF NOT EXISTS (
        SELECT 1 FROM fermate
        WHERE stazione = current_fermata
    ) THEN
        SET err_msg = CONCAT('Fermata non trovata: ', current_fermata);
        SIGNAL SQLSTATE '45001' SET MESSAGE_TEXT = err_msg;
    END IF;

    SELECT citta, provincia
    INTO citta_, provincia_
    FROM fermate
    WHERE stazione = current_fermata;

    INSERT INTO include (idTratta, stazione, citta, provincia, numeroFermata)
    VALUES (max_id, current_fermata, citta_, provincia_, counter);

    SET start = start + length + 1;
    SET counter = counter + 1;

    IF done THEN
        LEAVE cursor_loop;
    END IF;
```

```
    END LOOP;

    COMMIT;
END ;;
```

Procedura new_registra_corsa

Questa procedura consente di registrare una nuova corsa ferroviaria, inserendo i dettagli relativi a:

- Treno e lavoratori assegnati
- Fermate e orari (formattati da stringa testuale)
- Verifica della disponibilità dei lavoratori e del treno

Il livello di isolamento `REPEATABLE READ` viene scelto per assicurare la coerenza dei dati letti nella stessa transazione (es. massimo idCorsa, disponibilità treno e personale). La procedura è chiamata frequentemente (circa 75 volte al giorno), quindi richiede un buon equilibrio tra affidabilità e performance.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `new_registra_corsa`(
    IN var_macchinista VARCHAR(16),
    IN var_capotreno VARCHAR(16),
    IN var_treno VARCHAR(4),
    IN var_data DATE,
    IN var_idTratta INT,
    IN var_orari TEXT
)
BEGIN
    DECLARE var_idCorsa INT;
    DECLARE str_orario TEXT DEFAULT '';
    DECLARE orarioPartenza TIME;
    DECLARE orarioArrivo TIME;
    DECLARE numeroFermata INT DEFAULT 1;
    DECLARE primo_orario TIME;
    DECLARE ultimo_orario TIME;
    DECLARE a INT DEFAULT 1;
    DECLARE total_orari INT;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;
```

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
START TRANSACTION;

-- Rimuovere caratteri non numerici di sicurezza
SET var_orari = REPLACE(REPLACE(REPLACE(REPLACE(var_orari, '[', ''), ']', ''), ' ', ''), '(', '');

-- Calcolare il numero totale di orari usando il separatore ';'
SET total_orari = LENGTH(var_orari) - LENGTH(REPLACE(var_orari, ';', '')) + 1;

-- Estrai primo e ultimo orario usando ';'
SET primo_orario = SUBSTRING_INDEX(var_orari, ';', 1);
SET ultimo_orario = SUBSTRING_INDEX(var_orari, ';', -1);

IF (`DB_ferroviario`.controlla_lavoratori_treno(var_macchinista, var_capotreno, var_treno) IS FALSE) THEN
SIGNAL SQLSTATE '45025' SET MESSAGE_TEXT = 'Questo treno ha assegnati diversi lavoratori';
END IF;

IF (`DB_ferroviario`.controllo_treno(var_treno, var_data, primo_orario, ultimo_orario) IS TRUE) THEN
SIGNAL SQLSTATE '45020' SET MESSAGE_TEXT = 'Il treno è già impegnato in un altra corsa.';
END IF;

IF (`DB_ferroviario`.controllo_lavoratore(var_macchinista, var_capotreno, var_data, primo_orario, ultimo_orario) IS TRUE) THEN
SIGNAL SQLSTATE '45021' SET MESSAGE_TEXT = 'Il lavoratore è già impegnato in un altra corsa.';
END IF;

-- Genera nuovo idCorsa
SELECT IFNULL(MAX(idCorsa), 0) + 1 INTO var_idCorsa FROM corsa;
```

```
-- Inserisci la corsa
    INSERT INTO corsa (idCorsa, dataCorsa, idTratta, matricolaTreno, CFLavoratore1, CFLavoratore2)
VALUES (var_idCorsa, var_data, var_idTratta, var_treno, var_macchinista, var_capotreno);

-- Parsing orari con separatore ';'
WHILE_LOOP: WHILE LENGTH(var_orari) > 0 DO
  SET str_orario = TRIM(SUBSTRING_INDEX(var_orari, ';', 1));

  IF INSTR(str_orario, '-') > 0 THEN
    SET orarioPartenza = STR_TO_DATE(SUBSTRING_INDEX(str_orario, '-', 1), '%H:%i');
    SET orarioArrivo = STR_TO_DATE(SUBSTRING_INDEX(str_orario, '-', -1), '%H:%i');

    --  Controlla solo se entrambi sono diversi da 00:00:00
    IF orarioPartenza != '00:00:00' AND orarioArrivo != '00:00:00' THEN
      IF (`DB_ferroviario`.controllo_orari(orarioPartenza, orarioArrivo) IS FALSE) THEN
        SIGNAL SQLSTATE '45003' SET MESSAGE_TEXT = 'Orario di partenza è minore di quello di arrivo';
      END IF;
    END IF;

  ELSE
    IF a = 1 THEN
      SET orarioPartenza = STR_TO_DATE(str_orario, '%H:%i');
      SET orarioArrivo = STR_TO_DATE('00:00', '%H:%i');
    ELSEIF a = total_orari THEN
      SET orarioPartenza = STR_TO_DATE('00:00', '%H:%i');
      SET orarioArrivo = STR_TO_DATE(str_orario, '%H:%i');
    ELSE
      SET orarioPartenza = STR_TO_DATE(str_orario, '%H:%i');
      SET orarioArrivo = STR_TO_DATE('00:00', '%H:%i');
    END IF;
  END IF;
```

```
        INSERT INTO orari (numeroFermata, orarioPartenza, orarioArrivo, idCorsa, dataCorsa)
        VALUES (numeroFermata, orarioPartenza, orarioArrivo, var_idCorsa, var_data);

        SET var_orari = TRIM(SUBSTRING(var_orari, LENGTH(str_orario) + 2));
        SET numeroFermata = numeroFermata + 1;
        SET a = a + 1;
    END WHILE WHILE_LOOP;

    COMMIT;
END ;;
```

Procedura registra_prenotazione2

L'obiettivo di questa procedura è assegnare un posto disponibile in un vagone di un treno e registrare la prenotazione. Il livello di isolamento `REPEATABLE READ` è scelto per evitare letture sporche e inconsistenze, garantendo che durante l'esecuzione della procedura i dati letti non vengano modificati da altre transazioni.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `registra_prenotazione2`(
    IN tratta INT,
    IN var_partenza TIME,
    IN var_data DATE,
    IN CF VARCHAR(16)
)
BEGIN
    DECLARE codicePrenotazione VARCHAR(20);
    DECLARE conta INT;
    DECLARE vagoneC VARCHAR(10);
    DECLARE maxP INT;
    DECLARE cur_corsa INT;
    DECLARE cur_treno VARCHAR(4);
    DECLARE vagone_candidato VARCHAR(10);
    DECLARE vagone_trovato BOOLEAN DEFAULT FALSE;
    DECLARE posto_trovato INT;
    DECLARE ultimo_posto INT;
    DECLARE vagoni_cursor CURSOR FOR
        SELECT codice, maxPasseggeri
        FROM vagone
        WHERE matricolaTreno = cur_treno;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN

        ROLLBACK;
        RESIGNAL;
    END;
```

```
    SET TRANSACTION ISOLATION LEVEL repeatable read;
    START TRANSACTION;

-- mi trovo l'id della corsa
    SELECT corsa.idCorsa
    INTO cur_corsa
    FROM orari, corsa
    WHERE corsa.dataCorsa = orari.dataCorsa
      AND corsa.idCorsa = orari.idCorsa
      AND orarioPartenza = var_partenza
      AND corsa.dataCorsa = var_data
      AND corsa.idTratta = tratta;



-- mi trovo la matricola del treno
    SELECT matricolaTreno
    INTO cur_treno
    FROM corsa
    WHERE idCorsa = cur_corsa;

    OPEN vagoni_cursor;

    vagoni_loop: LOOP
        FETCH vagoni_cursor INTO vagoneC, maxP;

        IF vagone_trovato THEN
            LEAVE vagoni_loop;
        END IF;

        SELECT COUNT(*)
        INTO conta
        FROM prenotazione
        WHERE id_corsa = cur_corsa AND id_vagone = vagoneC;
```

```
        IF conta < maxP THEN
            SET vagone_candidato = vagoneC;
            LEAVE vagoni_loop;
        END IF;
    END LOOP;

    CLOSE vagoni_cursor;


    IF vagone_candidato IS NULL THEN
        ROLLBACK;
            SIGNAL SQLSTATE '45030' SET MESSAGE_TEXT = 'La corsa ha raggiunto la capienza
massima';
    END IF;

    SELECT MAX(postoTreno)
    INTO ultimo_posto
    FROM prenotazione
    WHERE id_vagone = vagone_candidato AND id_corsa = cur_corsa;

    IF ultimo_posto THEN
        SET posto_trovato = ultimo_posto + 1;
    ELSE
        SET ultimo_posto = 0;
        SET posto_trovato = 1;
    END IF;



    SET codicePrenotazione = SUBSTRING(MD5(CONCAT(UUID(), RAND())), 1, 20);

        INSERT INTO prenotazione (codicePrenotazione, postoTreno, CFAcquirente, matricolaTreno,
id_vagone, id_corsa, data_)
```

```
    VALUES (codicePrenotazione, posto_trovato, CF, cur_treno, vagone_candidato, cur_corsa,
var_data);

  COMMIT;
END ;;
```

Procedura report_turni

La procedura è progettata per generare un report richiesto da un lavoratore, mostrando i dettagli delle corse in cui è coinvolto (come macchinista o capotreno). Il livello di isolamento scelto è `READ UNCOMMITTED`, poiché i dati vengono utilizzati solo a scopo di lettura e eventuali letture inconsistenti non hanno conseguenze critiche.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `report_turni`(
        In CF VARCHAR(16)
)
BEGIN


        DECLARE exit handler for sqlexception
   BEGIN
      ROLLBACK;
      RESIGNAL;
   END;


   SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
   START TRANSACTION;


SELECT
   idCorsa,
   dataCorsa,
   matricolaTreno,
   MAX(orarioPartenza) AS orarioPartenza,
   MAX(orarioArrivo) AS orarioArrivo
FROM
```

```
   (SELECT

      corsa.idCorsa,

      corsa.dataCorsa,

      corsa.matricolaTreno,

         CASE WHEN orari.orarioArrivo = '00:00:00' THEN orari.orarioPartenza ELSE NULL END
AS orarioPartenza,

         CASE WHEN orari.orarioPartenza = '00:00:00' THEN orari.orarioArrivo ELSE NULL END
AS orarioArrivo

   FROM

      orari

   JOIN

      corsa ON corsa.dataCorsa = orari.dataCorsa AND corsa.idCorsa = orari.idCorsa

   WHERE

      (corsa.CFLavoratore1 = CF OR corsa.CFLavoratore2 = CF)

   ) AS combined_results

GROUP BY

   idCorsa, dataCorsa, matricolaTreno

ORDER By(dataCorsa);



commit;

END ;;
```

Procedura registrazione

La procedura è progettata per permettere ai vari utenti di registrarsi nel sistema. Controlla che il codice fiscale e la carta di credito siano validi prima di inserire i dati nel database.

```
CREATE PROCEDURE registrazione(
    IN p_codiceFiscale VARCHAR(16),
    IN p_nome VARCHAR(50),
    IN p_cognome VARCHAR(50),
    IN p_dataDiNascita DATE,
    IN p_numeroCartaDiCredito BIGINT,
    IN p_username VARCHAR(45),
    IN p_password VARCHAR(45),
    IN p_ruolo INT,
    IN p_ruoloLavoratore VARCHAR(45)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
    START TRANSACTION;

    IF p_ruolo = 1 THEN
        -- Acquirente: controllo CF e carta
```

```
    IF DB_ferroviario.controllo_cf(p_codiceFiscale) IS FALSE THEN
        SIGNAL SQLSTATE '45001' SET MESSAGE_TEXT = 'Il codice fiscale inserito non è valido.';
    END IF;
    IF DB_ferroviario.controllo_carta_di_credito(p_numeroCartaDiCredito) IS FALSE THEN
      SIGNAL SQLSTATE '45002' SET MESSAGE_TEXT = 'La carta di credito non è valida.';
    END IF;

    INSERT INTO acquirente (CF, cognome, nome, dataDiNascita, numeroCartaCredito, username, password)
      VALUES (p_codiceFiscale, p_cognome, p_nome, p_dataDiNascita, p_numeroCartaDiCredito, p_username, MD5(p_password));

  ELSEIF p_ruolo = 2 THEN
    -- Gestore del servizio
    INSERT INTO gestoredelservizio (username, password)
    VALUES (p_username, MD5(p_password));

  ELSEIF p_ruolo = 3 THEN
    -- Lavoratore (ha anche CF, data, ruoloLavoratore)
    IF DB_ferroviario.controllo_cf(p_codiceFiscale) IS FALSE THEN
        SIGNAL SQLSTATE '45003' SET MESSAGE_TEXT = 'Il codice fiscale inserito non è valido.';
    END IF;

    INSERT INTO lavoratore (CF, nome, cognome, dataNascita, username, password, ruolo)
        VALUES (p_codiceFiscale, p_nome, p_cognome, p_dataDiNascita, p_username, MD5(p_password), p_ruoloLavoratore);
```

```
    ELSEIF p_ruolo = 4 THEN
        -- Manutentore
        INSERT INTO manutentore (username, password)
        VALUES (p_username, MD5(p_password));

    ELSE
        SIGNAL SQLSTATE '45004' SET MESSAGE_TEXT = 'Ruolo non valido.';
    END IF;

    COMMIT;
END;;
```

Procedura login

La procedura è progettata per permettere agli utenti di eseguire il login nel sistema. La variabile `var_role` è usata per identificare il ruolo dell'utente dopo il login (acquirente, gestore, lavoratore, manutentore).

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `login`(in var_username
                                VARCHAR(45), in var_password VARCHAR(45), out var_role INT)
BEGIN


-- var_role = 0 --> login
-- var_role = 1 --> acquirente
-- var_role = 2 --> gestore
-- var_role = 3 --> lavoratore
-- var_role = 4 --> manutentore


            set var_role = 0;
    if exists(
                select * from acquirente
                WHERE username = var_username AND
                password = md5(var_password))
            then set var_role = 1;
    end if;


    if exists(
                select * from gestoredelservizio
                WHERE username = var_username AND
                password =  md5(var_password))
            then set var_role = 2;
```

```
    end if;

    if exists(
                select * from lavoratore
                WHERE username = var_username AND
                password =  md5(var_password))
        then set var_role = 3;
    end if;

    if exists(
                select * from manutentore
                WHERE username = var_username AND
                password =  md5(var_password))
        then set var_role = 4;
    end if;

END ;;
```

Procedura registra_manutenzione

Questa procedura consente a un manutentore di registrare un intervento di manutenzione su un treno. È stato scelto il livello di isolamento READ UNCOMMITTED poiché vengono solo aggiunti nuovi record e non è necessario bloccare letture su altre tabelle.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `registra_manutenzione`(
        in var_matricola varchar(45),
   in var_data varchar(45),
   in descrizione varchar(200)
)
BEGIN

declare exit handler for sqlexception
   begin
                rollback;
      resignal;
        end;

        set transaction isolation level read uncommitted;
        start transaction;

        insert into storico_manutenzione values (var_matricola, var_data,descrizione);

   commit;

END ;;
```

Procedura visualizza_lavoratori

La procedura è progettata per visualizzare i lavoratori in base al ruolo o alla matricola del treno. Il liello di isolamento scelto è `REPEATABLE READ` per garantire che i dati letti rimangano consistenti durante la transazione, evitando letture sporche.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `visualizza_lavoratori`(
    IN var_matricola VARCHAR(4),
    IN a INT
)
BEGIN
    DECLARE exit handler for sqlexception
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    SET TRANSACTION ISOLATION LEVEL repeatable read;
    START TRANSACTION;

    IF a = 0 THEN
        SELECT CF
        FROM lavoratore
        WHERE ruolo = 'capotreno';
    ELSEIF a = 1 THEN
        SELECT CF
        FROM lavoratore
        WHERE ruolo = 'macchinista';
    ELSEIF a = 2 THEN
        SELECT CFMacchinista
        FROM treno
```

```
        WHERE matricola = var_matricola;
    ELSEIF a = 3 THEN
        SELECT CFCapotreno
        FROM treno
        WHERE matricola = var_matricola;
    END IF;

    COMMIT;
END ;;
```

Procedura CF_Acquirenti

La procedura è progettata per leggere il codice fiscale (CF) dell'acquirente loggato all'applicazione. Il livello di isolamento scelto è `READ COMMITTED` per evitare letture sporche, garantendo che i dati letti siano consistenti, ma senza esagerare con l'isolamento, come sarebbe con `REPEATABLE READ` o `SERIALIZABLE`.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `CF_Acquirenti`(
        IN var_username VARCHAR(20)
)
BEGIN

declare exit handler for sqlexception
   begin
                rollback;
      resignal;
         end;

   set transaction isolation level read committed;
         start transaction;

   select CF
   from acquirente
   WHERE username=var_username;

   commit;

END ;;
```

Procedura CF_Lavoratori

La procedura è progettata per leggere il codice fiscale (CF) del lavoratore loggato all'applicazione. Il livello di isolamento scelto è `READ COMMITTED` per garantire che non vengano fatte letture sporche, mantenendo al contempo un buon livello di concorrenza.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `CF_Lavoratori`(
        IN var_username VARCHAR(20)
)
BEGIN

declare exit handler for sqlexception
   begin
                rollback;
      resignal;
        end;

   set transaction isolation level read committed;
        start transaction;

   select CF
   from lavoratore
   WHERE username=var_username;

   commit;

END ;;
```

Procedura visualizza_fermate

La procedura è progettata per visualizzare le fermate, che successivamente potrebbero essere utilizzate per registrare una nuova tratta. Il livello di isolamento scelto è `REPEATABLE READ` per garantire che i dati letti non siano soggetti a modifiche durante la transazione, evitando anomalie come letture sporche o inconsistenti, ad eccezione di inserimenti fantasma.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `visualizza_fermate`()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

  SELECT stazione, citta, provincia
  FROM fermate;

END ;;
```

Procedura visualizza_orari

La procedura è progettata per visualizzare gli orari delle corse per una data e una tratta specifica. Il livello di isolamento scelto è `REPEATABLE READ` per evitare anomalie come letture sporche o inconsistenti, mantenendo la consistenza dei dati letti, che verranno utilizzati successivamente per registrare nuove informazioni in altre tabelle.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `visualizza_orari`(
   IN deta DATE,
   IN id_Tratta INT
   )
BEGIN

DECLARE exit handler for sqlexception
   BEGIN
      ROLLBACK;
      RESIGNAL;
   END;

   SET TRANSACTION ISOLATION LEVEL repeatable read;
   START TRANSACTION;



         SELECT
   idCorsa,
   dataCorsa,
   matricolaTreno,
   MAX(orarioPartenza) AS orarioPartenza,
   MAX(orarioArrivo) AS orarioArrivo
FROM
   (SELECT
      corsa.idCorsa,
      corsa.dataCorsa,
      corsa.matricolaTreno,
         CASE WHEN orari.orarioArrivo = '00:00:00' THEN orari.orarioPartenza ELSE NULL END
AS orarioPartenza,
```

```
        CASE WHEN orari.orarioPartenza = '00:00:00' THEN orari.orarioArrivo ELSE NULL END
AS orarioArrivo
   FROM
      orari
   JOIN
      corsa ON corsa.dataCorsa = orari.dataCorsa AND corsa.idCorsa = orari.idCorsa
   WHERE
      ( orari.dataCorsa=deta AND corsa.idTratta=id_Tratta)
   ) AS combined_results
GROUP BY
   idCorsa, dataCorsa, matricolaTreno
ORDER BY
   dataCorsa;

   commit;
END ;;
```

Procedura visualizza_prenotazione

La procedura è progettata per visualizzare le prenotazioni fatte dagli utenti. Il livello di isolamento scelto è `READ COMMITTED` per evitare letture sporche, garantendo che i dati letti siano consistenti e non possano essere modificati da altre transazioni durante la lettura.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `visualizza_prenotazione`(
        IN Cf VARCHAR(16)
)
BEGIN


declare exit handler for sqlexception
   begin
                rollback;
      resignal;
        end;


   set transaction isolation level read committed;
        start transaction;


          SELECT prenotazione.matricolaTreno, prenotazione.id_vagone, prenotazione.data_, prenotazione.postoTreno, MIN(orari.orarioPartenza) AS minOrarioPartenza
FROM prenotazione
JOIN corsa ON prenotazione.id_corsa = corsa.idCorsa AND prenotazione.data_ = corsa.dataCorsa
JOIN orari ON corsa.idCorsa = orari.idCorsa AND corsa.dataCorsa = orari.dataCorsa
WHERE prenotazione.CFAcquirente = Cf AND orari.orarioPartenza != 0
GROUP BY corsa.idCorsa, corsa.dataCorsa, prenotazione.matricolaTreno, prenotazione.id_vagone, prenotazione.data_, prenotazione.postoTreno;



   commit;
```

END ;;

Procedura visualizza_tratte

La procedura è progettata per visualizzare le tratte disponibili al momento dell'acquisto di una nuova prenotazione. Il livello di isolamento scelto è `REPEATABLE READ` per garantire che i dati letti non possano essere modificati da altre transazioni durante la lettura, prevenendo letture inconsistenti, ma lasciando la possibilità di inserimenti fantasma.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `visualizza_tratte`()

BEGIN



        DECLARE exit handler for sqlexception

    BEGIN

        ROLLBACK;

        RESIGNAL;

    END;


    SET TRANSACTION ISOLATION LEVEL repeatable read;

    START TRANSACTION;



    SELECT numeroFermata, idTratta, stazione

    FROM include

    ORDER BY idTratta,numeroFermata;



END ;;
```

Procedura visualizza_treni

La procedura è progettata per visualizzare i treni presenti nel sistema. Il livello di isolamento scelto è `REPEATABLE READ` per garantire che i dati letti durante la transazione non possano essere modificati da altre transazioni, evitando letture inconsistenti, ma lasciando la possibilità di inserimenti fantasma.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `visualizza_treni`()

BEGIN


        DECLARE exit handler for sqlexception

   BEGIN

      ROLLBACK;

      RESIGNAL;

   END;


   SET TRANSACTION ISOLATION LEVEL repeatable read;

   START TRANSACTION;


   select matricola

   from treno




   commit;


END ;;
```